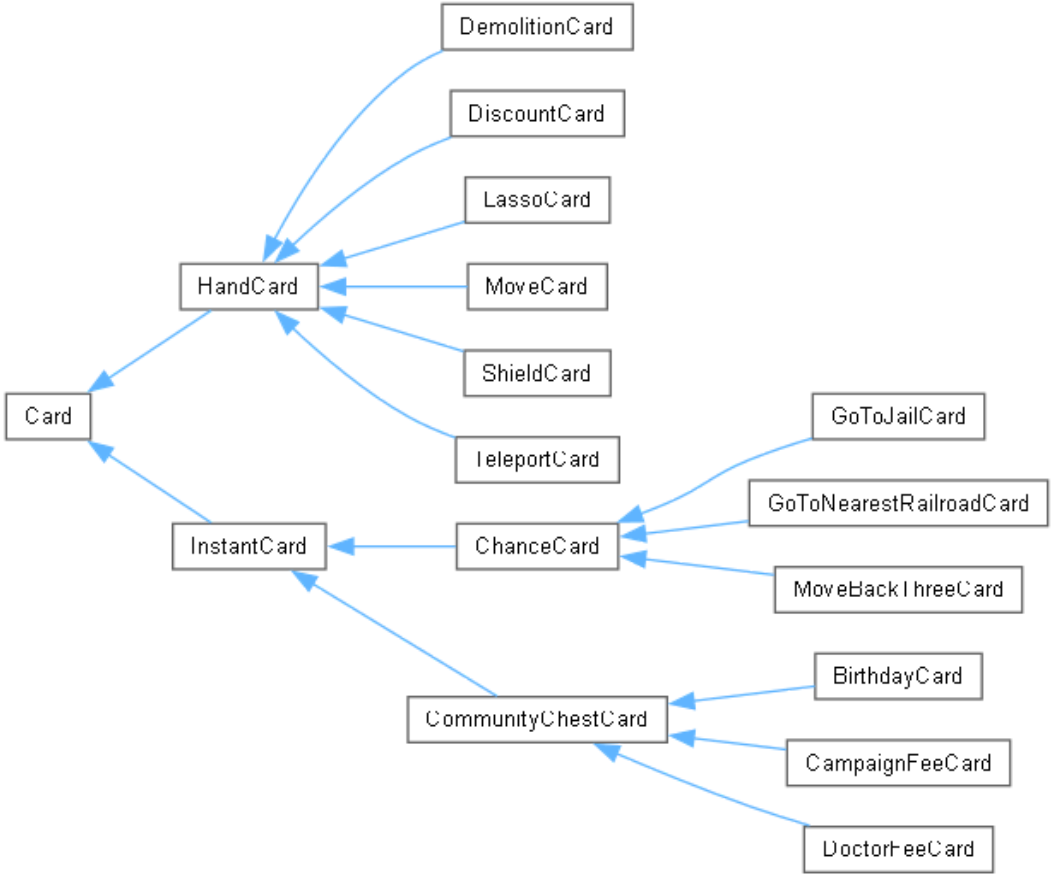
Laporan M2 Tugas Besar 1
IF2010 Pemrograman Berorientasi Objek 2025/2026
Nimonspoli

Kode Kelompok : SHT
Nama Kelompok : Sua Huan Te Nang

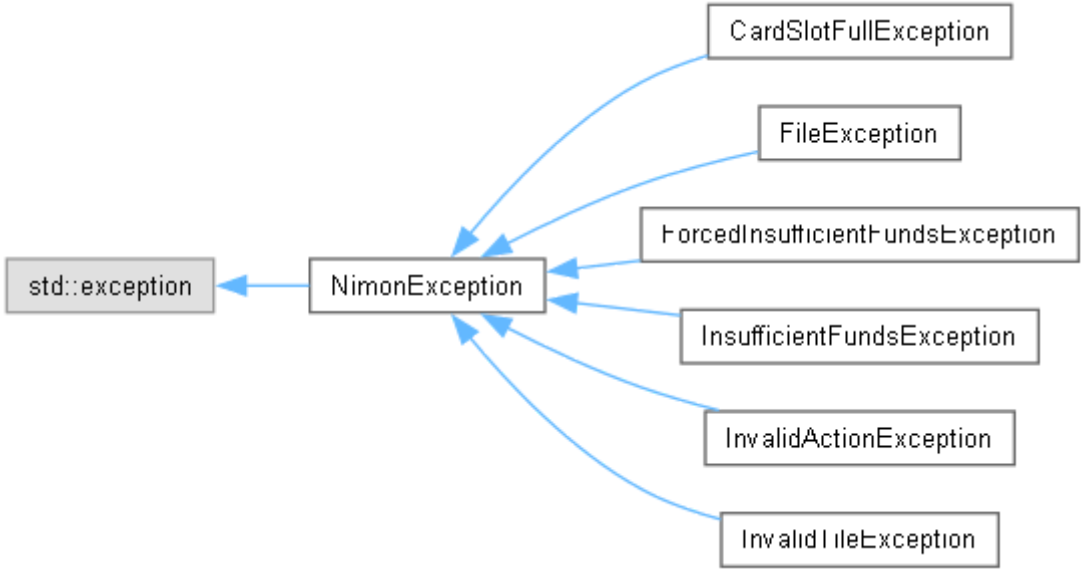
1. 13524010 / Audric Yusuf Maynard Simatupang
2. 13524022 / Ishak Palentino Napitupulu
3. 13524052 / Raynard Fausta
4. 13524062 / Nathan Edward Christofer Marpaung
5. 13524112 / Richard Samuel Simanullang

SHT - Sua Huan Te Nang

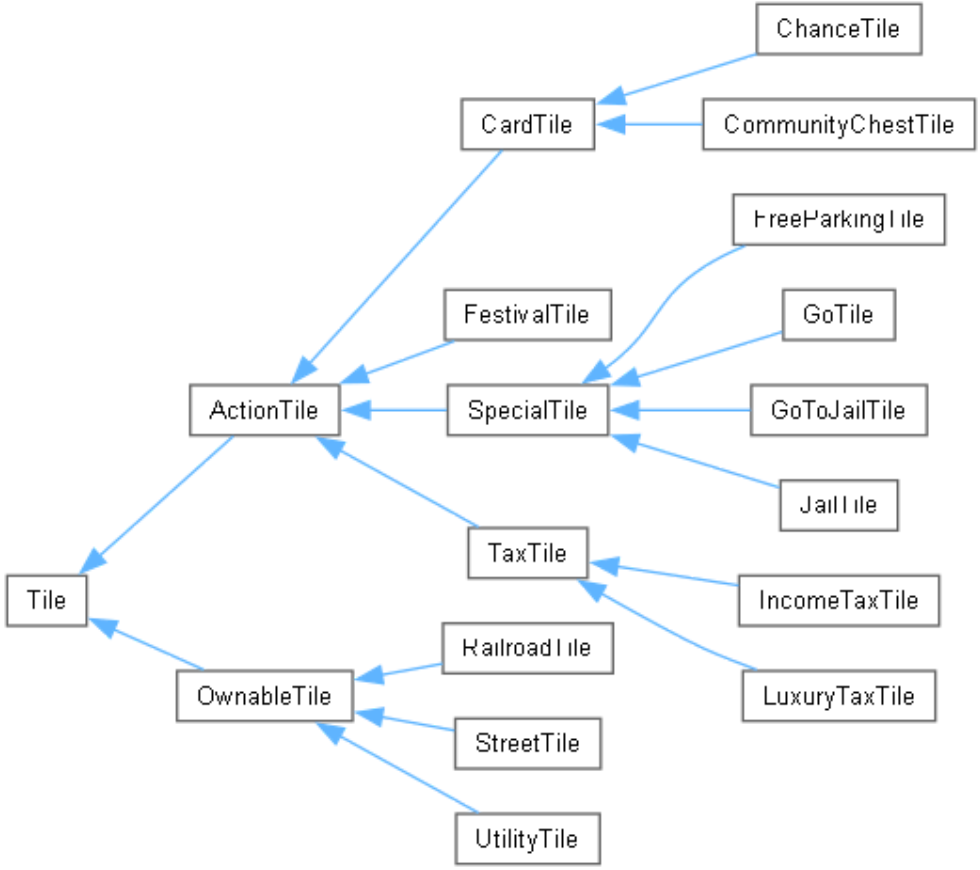
1. Diagram Kelas



Gambar 1.1 Diagram Kelas (Card)



Gambar 1.2 Diagram Kelas (Exception)



Gambar 1.3 Diagram Kelas (Tile)

Gambar 1.4 Diagram Kelas (Lainnya)

List kelas dan diagram hierarchy setiap kelas dapat dilihat di link yang dilampirkan pada lampiran. Link berisi file html dari hasil analisis doxygen.

2. Deskripsi Kelas

2.1. Kelas MainApplication

Kelas ini bertindak sebagai pembungkus proses awal program di main.cpp. Method run memilih mode CLI, GUI, atau keluar, sedangkan helper private yang menangani input mode. Class ini berelasi dengan Application untuk mode CLI dan RaylibGuiApp untuk mode GUI.

2.2. Application

Kelas Application menjadi titik masuk untuk aplikasi CLI. Kelas ini berkoordinasi dengan Game, UIManager, serta controller CLI untuk menjalankan alur permainan. Atributnya adalah instansi Game, UIManager, dan GameController. Method run() merupakan entry point utama yang menampilkan menu dan mengelola pilihan pengguna. Method showMainMenu() dan getMainMenuChoice() menampilkan dan membaca pilihan dari menu utama. Method handleNewGame() menginisialisasi dan memulai permainan baru setelah mengumpulkan konfigurasi awal dari pengguna melalui UIManager. Method handleLoadGame() memulihkan permainan dari file tersimpan. Method exitApplication() menangani proses keluar dengan bersih. Application memiliki ketiga komponen utama (model, view, controller) dan bertanggung jawab menghubungkan ketiganya saat inisialisasi. Kelas ini memisahkan setup awal dari logika permainan yang ditangani oleh GameController.

2.3. UIManager

Kelas UIManager adalah antarmuka pengguna berbasis teks (CLI) yang menangani seluruh input dan output ke terminal. UIManager tidak mengandung logika permainan namun hanya memformat dan menampilkan data yang diterima dari controller, dan mengembalikan input pengguna dalam format yang dapat diproses oleh controller.

2.4. RaylibGuiApp

Kelas RaylibGuiApp adalah titik masuk untuk aplikasi GUI berbasis Raylib. Atributnya meliputi toolkit_, controller_, dan atribut renderer lainnya. Method run() menjalankan loop utama Raylib untuk memperbarui state, kemudian memanggil renderer yang sesuai berdasarkan state aplikasi saat ini (menu, gameplay, atau overlay). RaylibGuiApp memisahkan loop rendering dari logika permainan.Pemisahan ini memungkinkan logika permainan diuji secara terpisah dari rendering grafis.

2.5. GameController

Kelas GameController mengatur alur utama permainan berbasis CLI. Atribut utamanya adalah referensi ke Game (model) dan UIManager (view), serta kepemilikan CommandController dan TileController. Atribut rolledThisTurn, diceRolledThisTurn, dan resumeLoadedTurnPending mencatat status giliran yang sedang berlangsung.

Method runGameLoop() adalah loop utama permainan yang berjalan hingga kondisi menang terpenuhi. Method runTurn() mengeksekusi satu giliran penuh untuk pemain aktif. Method handleStartTurn() menginisialisasi giliran baru (memberikan kartu, menentukan aksi awal), dan handleEndTurn() menyelesaikan giliran serta memeriksa kondisi menang. Method canSaveNow() memvalidasi apakah permainan dapat disimpan pada saat ini (misalnya tidak di tengah lelang atau kebangkrutan), dan isCommandValidThisTurn() memeriksa apakah perintah tertentu valid untuk kondisi giliran saat ini.

GameController mendelegasikan pemrosesan perintah ke CommandController dan penanganan pendaratan tile ke TileController, menjaga tanggung jawabnya tetap terfokus pada koordinasi alur giliran.

2.6. CommandController

Kelas CommandController memproses dan mengeksekusi perintah-perintah yang diberikan pemain selama giliran berlangsung. Atributnya adalah referensi ke Game dan UIManager. Method processCommand() menerima string perintah dan mendelegasikannya ke handler yang sesuai.

Handler spesifik mencakup: handleRollDice() (melempar dadu dan memindahkan pemain), handleSetDice() (mengatur nilai dadu manual untuk debugging), handlePrintBoard() (menampilkan papan), handlePrintDeed() (menampilkan sertifikat properti), handlePrintProperties() (menampilkan daftar properti pemain), handleCheckMoney() (menampilkan saldo), handleMortgage() (menggadaikan properti), handleRedeem() (menebus properti yang digadaikan), handleBuild() (membangun rumah/hotel), handleSave() (menyimpan permainan), handlePrintLog() (menampilkan riwayat aksi), handleUseCard() (menggunakan kartu tangan), dan handleHelp() (menampilkan bantuan perintah).

CommandController berinteraksi dengan PropertyManager, CardManager, dan SaveLoadManager melalui Game, serta menggunakan UIManager untuk input/output. Untuk pemain komputer, keputusan dikonsultasikan ke ComputerDecisionMaker.

2.7. TileController

Kelas TileController menangani seluruh event yang dipicu oleh pendaratan pemain di petak tertentu. Atribut pendingDebtAmount, pendingCreditor, dan pendingDebtToBank menyimpan konteks utang yang mungkin timbul dari pendaratan. Method utama resolveLanding() menerima tile yang didarati dan mendelegasikan ke handler yang spesifik.

Handler-handler spesifik meliputi: handleStreetPurchase() (menawarkan pembelian properti atau memicu lelang), handleAuction() (menjalankan sesi lelang melalui AuctionManager), handleIncomeTax() (memproses pilihan pajak penghasilan), handleLuxuryTax() (memproses pajak kemewahan), handleFestival() (menangani pemilihan properti untuk diaktifkan festival), handleForceDrop() (memaksa pemain membuang kartu tangan berlebih), handleJailTurn() (mengelola pilihan pemain di penjara), dan handleBankruptcy() yang dipanggil oleh triggerBankruptcy() untuk memulai sesi likuidasi.

TileController berinteraksi dengan AuctionManager, BankruptcyManager, JailManager, FestivalManager, dan CardManager melalui Game, serta menggunakan UIManager untuk interaksi dengan pengguna.

2.8. CardController

Kelas CardController bertanggung jawab atas proses penarikan dan penerapan kartu. Method drawAndApplyChanceCard() menarik kartu dari dek Kesempatan melalui CardManager dan langsung memanggil apply() pada kartu yang diambil. Method drawAndApplyCommunityChestCard() melakukan hal serupa untuk dek Kas Umum. Method useHandCard() memvalidasi dan mengeksekusi penggunaan kartu tangan yang dipilih pemain dari inventori tangannya.

CardController berinteraksi dengan CardManager untuk operasi dek dan dengan Player untuk akses inventori kartu tangan. Penerapan kartu melalui apply(Game&, Player&) memungkinkan setiap jenis kartu mengeksekusi efeknya sendiri tanpa CardController harus mengetahui detail implementasi setiap kartu.

2.9. GUIGameController

Kelas GUIGameController setara dengan GameController untuk antarmuka grafis berbasis Raylib. Atribut utamanya adalah appState_ (seluruh state UI), backendGame_ (model permainan), controllerContext_ (konteks bersama untuk sub-controller GUI). GUIGameController menjembatani antara GUI renderer (yang hanya membaca AppState) dan model backend, memastikan kedua sisi selalu tersinkronisasi.

2.10. GUIControllerContext

Kelas GUIControllerContext adalah objek konteks yang dibagikan kepada seluruh sub-controller GUI. Atributnya mencakup referensi ke appState_, backendGame_, flag-flag status giliran, referensi ke kartu pending, dan berbagai fungsi callback. Kelas ini menghindari duplikasi state dan referensi dengan mengumpulkan semua dependensi bersama dalam satu objek yang diteruskan ke GUICardController, GUICommandController, dan GUITileController.

2.11. GUICommandController

Kelas ini mengelola command global GUI seperti start turn, roll dice, manual dice, save, overlay, shortcut keyboard, dan end turn. Class ini memakai `GUIControllerContext` untuk mengubah state, memanggil manager backend, menulis log, dan menampilkan toast.

2.12. GUITileController

GUITileController adalah sub-controller GUI yang menangani aksi-aksi yang berhubungan dengan petak atau tile pada papan permainan. Class ini dipakai ketika pemain mendarat di suatu petak, memilih detail tile, membeli properti, mengikuti lelang, membayar pajak, masuk penjara, melakukan build, gadai, tebus, atau likuidasi aset. Jadi, meskipun input awalnya berasal dari GUI, logic untuk menentukan aksi tile diarahkan ke GUITileController agar tidak menumpuk di GUIGameController.

2.13. GUICardController

GUICardController adalah sub-controller GUI yang menangani seluruh aksi yang berhubungan dengan kartu. Class ini digunakan untuk mengambil kartu Chance atau Community Chest, menampilkan kartu pada overlay, menyimpan kartu kemampuan ke tangan pemain, menggunakan kartu kemampuan, dan membuang kartu jika jumlah kartu melebihi batas. Dengan adanya GUICardController, semua fitur kartu dipisahkan dari logic tile dan command umum sehingga struktur GUI menjadi lebih rapi.

2.14. Theme

Theme digunakan untuk menyimpan kumpulan warna yang digunakan pada GUI. Contohnya adalah warna background, teks, tombol, panel, warna pemain, warna sukses, dan warna error. Dengan adanya Theme, warna pada tampilan GUI dapat diatur secara terpusat sehingga tampilan menjadi lebih konsisten.

2.15. UiToolkit

UiToolkit adalah kelas bantuan yang digunakan untuk menggambar komponen GUI yang sering dipakai. Contohnya adalah button, panel, modal, badge, text field, background, dan format uang. Dengan UiToolkit, renderer tidak perlu menulis ulang kode untuk komponen UI yang sama.

2.16. MenuScreenRenderer

MenuScreenRenderer bertanggung jawab untuk menggambar tampilan menu pada aplikasi. Bagian ini menangani layar Main Menu, New Game, Load Game, dan Tutorial. Dengan class ini, tampilan menu dipisahkan dari tampilan gameplay.

2.17. GameplayScreenRenderer

GameplayScreenRenderer adalah renderer utama saat permainan berlangsung. Class ini menggambar board, tile, posisi pemain, panel pemain aktif, tombol aksi, log singkat, dan toast. Input sederhana seperti klik tile atau tombol juga dibaca di sini, lalu diteruskan ke controller.

2.18. OverlayRenderer

OverlayRenderer digunakan untuk menggambar popup atau modal yang muncul di atas tampilan utama. Contohnya adalah popup beli properti, auction, kartu, pajak, jail, save game, logs, dan game over. Dengan OverlayRenderer, semua popup dapat ditangani secara terpusat.

2.19. CardInfo

CardInfo adalah data tampilan untuk kartu. Data ini berisi informasi seperti nama kartu, jenis kartu, deskripsi efek, warna aksen, dan detail lain yang dibutuhkan agar kartu dapat ditampilkan pada GUI.

2.20. TileInfo

TileInfo adalah data tampilan untuk setiap petak pada board. Data ini menyimpan informasi seperti nama petak, kode petak, jenis petak, warna kategori, pemilik, status kepemilikan, harga, dan informasi visual lain yang dibutuhkan oleh renderer.

2.21. TileDeedInfo

TileDeedInfo digunakan untuk menyimpan informasi detail akta properti. Data ini dipakai saat GUI menampilkan detail properti seperti harga beli, nilai gadai, harga sewa tiap level, biaya rumah, biaya hotel, dan status kepemilikan.

2.22. PlayerInfo

PlayerInfo adalah data tampilan untuk pemain. Data ini berisi nama pemain, jumlah uang, posisi di papan, warna pemain, status seperti active atau jailed, daftar properti, dan kartu yang dimiliki.

2.23. SaveSlot

SaveSlot digunakan untuk menyimpan informasi mengenai slot save game. Data ini membantu GUI menampilkan daftar save yang tersedia, seperti nama file save, label save, atau ringkasan singkat dari state permainan yang tersimpan.

2.24. LogItem

LogItem merepresentasikan satu entri log transaksi dalam permainan. Setiap log biasanya berisi turn, nama pemain, jenis aksi, dan detail kejadian, misalnya membeli properti, membayar sewa, melempar dadu, atau menggunakan kartu.

2.25. Toast

Toast adalah notifikasi singkat yang muncul sementara pada layar. Toast digunakan untuk memberi feedback cepat kepada pemain, seperti uang tidak cukup, properti berhasil dibeli, kartu digunakan, atau dadu sudah dilempar.

2.26. AuctionState

AuctionState menyimpan data khusus selama proses lelang berlangsung. State ini mencatat properti yang sedang dilelang, bid tertinggi, pemain dengan bid tertinggi, giliran bidder saat ini, dan jumlah pemain yang melakukan pass.

2.27. OverlayState

OverlayState menyimpan kondisi overlay atau popup yang sedang aktif. Data ini menentukan jenis overlay yang sedang dibuka, tile yang sedang dipilih, kartu yang sedang ditampilkan, input pemain, dan state tambahan seperti AuctionState.

2.28. GameState

GameState adalah state gameplay yang sudah siap ditampilkan oleh GUI. Di dalamnya terdapat informasi seperti daftar tile pada board, daftar pemain, giliran aktif, hasil dadu terakhir, status turn, log permainan, dan toast yang sedang aktif.

2.29. AppState

AppState adalah state utama aplikasi GUI. State ini menyimpan kondisi keseluruhan aplikasi, seperti screen yang sedang aktif, GameState, OverlayState, input pada menu, daftar save slot, waktu animasi, dan status keluar aplikasi.

2.30. CenterLine

CenterLine adalah struktur kecil yang digunakan untuk membantu menyusun teks di area tengah board. Biasanya data ini dipakai untuk menampilkan nama tile, jenis tile, status, atau detail ringkas lain agar tampilan tengah papan lebih rapi.

2.31. Game

Kelas Game merupakan kelas utama yang mengatur keseluruhan permaina. Tanggung jawab utamanya adalah mengelola seluruh komponen permainan melalui komposisi dengan berbagai kelas manajer dan komponen inti. Kelas ini menyimpan seluruh state permainan melalui atribut GameContext, Dice, TurnManager, MovementManager, PropertyManager, AuctionManager, CardManager, JailManager, BankruptcyManager, SaveLoadManager, ConfigManager, LogManager, TaxManager, FestivalManager, dan WinConditionManager. Setiap manajer bertanggung jawab atas satu aspek spesifik permainan, sehingga Game berperan sebagai titik akses tunggal yang menghubungkan semua komponen tersebut.

Method startNewGame() menginisialisasi seluruh komponen permainan dari awal, sedangkan loadGame() memulihkan state dari file penyimpanan melalui SaveLoadManager. Method getCurrentPlayer() dan getBoard() menyediakan akses ke state aktif permainan. Relasi Game dengan kelas lain bersifat komposisi penuh — semua manajer dimiliki oleh Game dan diakses oleh controller. GameContext yang dimiliki oleh Game menjadi sumber kebenaran tunggal bagi semua manajer yang membutuhkan data pemain dan papan.

2.32. GameContext

Kelas GameContext berperan sebagai wadah (container) untuk menyimpan state inti permainan yang perlu diakses oleh berbagai komponen. Atribut currentTurn dan maxTurn mencatat progres putaran permainan, isRunning menandai apakah permainan sedang berlangsung, board menyimpan papan permainan, dan player menyimpan daftar seluruh pemain aktif.

Seluruh method pada kelas ini berupa getter dan setter (getCurrentTurn(), getMaxTurn(), isGameRunning(), getBoard(), getPlayers(), dan setter-setter yang sesuai). GameContext menjadi jembatan antara Game dan semua kelas manajer agar setiap manajer menerima referensi ke GameContext sehingga dapat mengakses Board dan Player tanpa harus bergantung langsung ke Game. GameContext memisahkan ketergantungan antar komponen dan menjaga desain tetap modular.

2.33. Board

Kelas Board merepresentasikan papan permainan Nimonspoli. Atribut utamanya tiles, vector shared_ptr<Tile> untuk seluruh petak, dan tileIndexByCode, map dari kode petak ke indeksnya untuk pencarian cepat. Method initializeBoard() membangun seluruh petak menggunakan data konfigurasi dari ConfigManager. Method getTile() dan getTileByCode() menyediakan akses ke petak berdasarkan indeks atau kode unik, sementara getNextPosition() menghitung posisi berikutnya setelah mempertimbangkan pergerakan melingkar papan.

Method spesifik seperti getStreetTileByColorGroup() mendukung logika pembangunan properti (mencari petak-petak dalam satu kelompok warna), countRailroadsOwned() dan countUtilitiesOwned() mendukung kalkulasi sewa, dan fixupOwnerPointers() memulihkan hubungan pointer pemilik setelah memuat permainan dari file. Board memiliki relasi komposisi dengan seluruh kelas turunan Tile dan menjadi acuan utama bagi MovementManager dan PropertyManager untuk navigasi dan manajemen properti.

2.34. Player

Kelas Player merepresentasikan seorang pemain dalam permainan, baik player biasa maupun komputer. Kelas ini menyimpan seluruh informasi yang relevan dengan satu pemain: username sebagai identitas, money sebagai saldo, position sebagai posisi di papan, status yang menentukan kondisi pemain, serta handCards berupa vector kartu yang sedang dipegang. Atribut tambahan seperti failedJailRolls, shieldActive, discountPercent, discountDuration, dan usedHandCardThisTurn mencatat efek sementara yang sedang aktif pada pemain.

Method moveTo() memindahkan posisi pemain di papan, method aritmetika operator+= dan operator-= memanipulasi saldo, dan isBankrupt() serta isJailed() mengecek kondisi pemain. Method effectiveCost() menghitung biaya setelah diskon aktif diterapkan, sedangkan decrementDiscountDuration() mengurangi durasi efek diskon setiap giliran. Player memiliki relasi agregasi dengan HandCard melalui

handCards, dan relasi asosiasi dengan OwnableTile. BankruptcyManager, PropertyManager, dan JailManager semuanya beroperasi langsung pada objek Player.

2.35. Dice

Kelas Dice bertanggung jawab mengelola mekanisme pelemparan dadu dalam permainan. Atribut die1 dan die2 menyimpan hasil masing-masing dadu, sedangkan manualMode memungkinkan pengujian dengan nilai dadu yang telah ditentukan sebelumnya. Method roll() menghasilkan nilai acak untuk kedua dadu saat mode normal aktif. Method setManual() mengaktifkan mode manual. Method getDie1(), getDie2(), getTotal(), dan isDouble() menyediakan akses ke hasil lemparan, serta isDouble() secara khusus penting karena lemparan ganda (double) dalam aturan Monopoli memungkinkan pemain melempar kembali atau dapat mengirim pemain ke penjara jika terjadi tiga kali berturut-turut.

Dice digunakan oleh TurnManager untuk mencatat apakah giliran saat ini sudah melempar dadu, dan oleh MovementManager untuk menentukan jarak pergerakan pemain

2.36. RentContext

Kelas RentContext merangkum informasi yang dibutuhkan untuk kalkulasi rent di tile tertentu. Kelas ini memiliki atribut ownerHasColorGroup yang menandai hubungan pemilik dan warna tile di permainan, railroadRent yang menyimpan sewa stasiun, utilityMultiplier untuk sewa utility, serta getter dan setter yang mengakses atribut tersebut. RentContext ini dibuat oleh PropertyManager setiap perhitungan sewa diperlukan dan diteruskan ke method kalkulasi rent pada tile yang bersangkutan.

2.37. ComputerDecisionMaker

Kelas ini merupakan utility class untuk seluruh logika keputusan komputer (com). Kelas ini berisi method yang mengatur randomisasi serta pengambilan keputusan spesifik yang mencakup berbagai situasi. Kelas ini digunakan oleh TileController, CommandController, serta manager lainnya untuk memisahkan logika com dari logika inti permainan.

2.38. ConfigManager

Kelas ConfigManager bertanggung jawab memuat dan menyediakan konfigurasi permainan dari file config. Terdapat inner class ActionTileConfig dan PropertyConfig yang membagi tugas dalam menyimpan konfigurasi. Atribut yang ada di kelas ini seperti

actionFileConfigs, propertyConfigs, initialBalance, dan lainnya menyimpan seluruh parameter permainan yang ada di file config. ConfigManager digunakan terutama saat Board::initializeBoard() membangun seluruh petak.

2.39. ConfigManager::ActionTileConfig

Kelas ini menyimpan konfigurasi tile aksi seperti kode, nama, deskripsi, dan tipe efek. Dipakai ConfigManager saat membangun board.

2.40. ConfigManager::PropertyConfig

Kelas ini menyimpan konfigurasi properti seperti kode, nama, harga, rent, mortgage, warna, dan biaya bangunan. Dipakai Board untuk membuat StreetTile, RailroadTile, dan UtilityTile.

2.41. PropertyManager

PropertyManager akan berinteraksi dengan Game dan GameContext untuk mengakses property yang ada dan aktif di dalam game, dan mengeksekusi setiap aksi di dalam game yang berhubungan dengan properti, seperti membeli dan menjual properti, mengecek color group dan membangun rumah/hotel, dan membayar uang sewa.

Method mortgageProperty() dan redeemProperty() mengubah status properti sambil mengalihkan uang ke/dari pemain. Method buildOnStreet() membangun rumah atau hotel pada petak jalan, dengan validasi bahwa monopoli warna dimiliki dan bangunan dibangun secara merata. Method sellBuildingsInColorGroup() menjual semua bangunan dalam satu kelompok warna ke bank. Method utilitas seperti canMortgage(), canBuild(), isMonopoly(), getOwnedProperties(), dan calculateTotalWealth() mendukung proses validasi dan pengambilan keputusan oleh controller dan Com. PropertyManager bergantung pada Board, Player, dan OwnableTile, serta berinteraksi dengan BankruptcyManager dalam proses likuidasi aset.

2.42. CardManager

Kelas CardManager mengelola seluruh dek kartu dalam permainan. Atribut chanceDeck, communityChestDeck, dan handDeck masing-masing adalah instansi CardDeck<T> yang menampung kartu sesuai jenisnya. Atribut forceDropActive menandai kondisi di mana pemain dipaksa membuang kartu tangan karena melebihi kapasitas.

Method initializeDecks() mengisi semua dek dengan kartu sesuai konfigurasi. Method drawChanceCard() dan ddrawCommunityChestCard() mengambil dan mengembalikan kartu untuk langsung diterapkan. Method drawHandCard() memberikan kartu tangan kepada pemain, sedangkan discardHandCard() memindahkan kartu bekas pakai ke discard pile. Method giveStartTurnCard() memberikan kartu tangan kepada pemain di awal giliran sesuai aturan. Method beginForceDrop() dan dropHandCard() menangani skenario pemain yang harus membuang kartu. Method useHandCard() mengaplikasikan efek kartu tangan yang dipilih pemain. Method updateTemporaryEffects() dipanggil setiap akhir giliran untuk mengurangi durasi efek sementara kartu. CardManager berelasi dengan CardDeck<T>, Player (pemegang kartu), dan HandCard/InstantCard.

2.43. CardDeck<T>

CardDeck<T> adalah template class yang mengelola dek kartu generik dengan mekanisme draw dan discard pile. Atribut drawPile dan discardPile masing-masing berupa vector shared_ptr<T>. Method addCard() menambahkan kartu ke dek, drawCard() mengambil kartu dari puncak draw pile, dan discardCard() memindahkan kartu ke discard pile. Method reshuffleDiscardIntoDrawPile() mengocok discard pile kembali ke draw pile saat draw pile kosong.

Kelas ini digunakan oleh CardManager dengan tiga instansiasi: CardDeck<ChanceCard>, CardDeck<CommunityChestCard>, dan CardDeck<HandCard>. Penggunaan template memungkinkan logika deck yang sama diterapkan untuk jenis kartu yang berbeda tanpa duplikasi kode.

2.44. AuctionManager

AuctionManager akan berinteraksi dengan Game dan GameContext untuk mengakses status game dan menginisiasikan auction jika dimulai dari game, seperti pengecekkan dan penaruhan bid dari player, dan memfinalisasi auction. Atribut currentBid dan highestBidder mencatat penawaran tertinggi saat ini, participants berisi pointer ke semua pemain yang terlibat, currentAuctionPlayerIndex menunjuk peserta yang giliran menawar, dan consecutivePasses melacak berapa kali berturut-turut peserta melewatkan giliran. Atribut auctionActive, hasAnyBid, currentTile, dan triggerPlayer menyimpan konteks lelang yang sedang berlangsung.

Method initializeAuction() memulai sesi lelang baru untuk properti tertentu, getCurrentAuctionPlayer() mengembalikan peserta yang sedang aktif, dan canBid()/placeBid() memvalidasi dan memproses penawaran. Method canPass()/pass() memungkinkan peserta melewatkan giliran menawar. Method isAuctionOver() memeriksa kondisi berakhirnya lelang (satu pemain tersisa atau semua melewatkan), dan finalizeAuction() mengeksekusi transfer kepemilikan dan pembayaran kepada pemenang. Method requiresForcedOpeningBid() menangani

aturan khusus di mana pemain yang memicu lelang harus membuka dengan harga minimal. AuctionManager berinteraksi langsung dengan Player dan OwnableTile, serta dikendalikan oleh TileController.

2.45. TaxManager

TaxManager akan berinteraksi dengan state player, dan memproses pembayaran pajak dari sebuah player berdasarkan amount yang diinput. Jika uang player tidak cukup, maka TaxManager akan melempar exception untuk menginisiasikan proses bangkrut pada player tersebut.

Method calculateTotalWealth() menghitung total kekayaan pemain yang digunakan sebagai dasar perhitungan pajak penghasilan persentase. Method processTaxPayment() mengeksekusi transaksi pembayaran pajak dari pemain ke bank. TaxManager berinteraksi dengan Player (mengambil saldo dan properti) dan Board (mendapatkan data tile pajak) untuk menyelesaikan kalkulasi dan transaksi.

2.46. MovementManager

MovementManager akan menghandle setiap pergerakan dan perubahan posisi dari player, dan menggerakkan player berdasarkan input parameter yang dimasukkan. MovementManager dapat menggerakkan player berdasarkan jumlah langkah yang diambil ataupun posisi baru di papan. MovementManager juga membantu menghandle pergerkan player menuju stasiun terdekat, ataupun menghandle pengecekkan apakah player melewati petak Go atau tidak.

Method passedGo() mendeteksi apakah pemain melewati petak Go dalam pergerakan dan memberikan gaji dari GoTile. Method resolveLanding() memanggil onLand() pada petak tujuan, kemudian meneruskan penanganan ke controller yang sesuai. MovementManager bergantung pada Board untuk menghitung posisi baru dan pada Player untuk memperbarui posisi. Relasi ini membuatnya menjadi penghubung antara fisik papan dan logika aksi yang dipicu oleh pergerakan.

2.47. JailManager

JailManager akan menghandle seluruh proses Jail yang terdapat pada game, yaitu mengirim sebuah player ke jail dengan menset status player menjadi Jailed, memproses pembayaran denda jail, mencoba roll dice untuk release dari jail, dan menset status player menjadi bebas dari jail.

Method beginJailDecision() dan resolveJailDecision() menangani alur interaksi antara pemain dan sistem saat pemain harus memilih tindakan di penjara. Jika pemain adalah komputer, keputusan didelegasikan ke ComputerDecisionMaker. JailManager beroperasi pada objek Player (mengubah status) dan berinteraksi dengan TurnManager untuk penanganan giliran pemain yang terpenjara.

2.48. TurnManager

TurnManager akan berinteraksi dengan state game dari Game dan GameContext, dan menghandle seluruh proses Turn pada game. TurnManager akan menginisiasi turn order di awal, menentukan awal turn dan akhir turn, sekaligus pengecekkan terhadap banyaknya dice double yang sudah dilakukan untuk player (digunakan untuk pengiriman player ke jail jika double terjadi 3 kali). TurnManager memiliki setter dan getter untuk membantu menset turnorder dan turncounter dari game.

Method initializeTurnOrder() menentukan urutan awal pemain, getCurrentPlayer() mengembalikan pemain yang sedang aktif, dan nextPlayer() menggeser giliran ke pemain berikutnya (dengan pengecekan skip untuk pemain bangkrut). Method startTurn() dan endTurn() mereset flag status giliran, registerDiceResult() mencatat hasil lemparan dadu termasuk penanganan lemparan ganda, dan canRollDice() memvalidasi apakah pemain diizinkan melempar dadu saat ini. Method updateTurnEffects() mengurangi durasi efek sementara pada pemain di akhir giliran. TurnManager berinteraksi erat dengan Player (untuk memeriksa status) dan dengan GameController/GUIGameController yang mengatur alur giliran.

2.49. SaveLoadManager

Kelas SaveLoadManager bertanggung jawab menyimpan dan memuat seluruh state permainan ke/dari file. Method saveGame() mengkoordinasikan seluruh proses penyimpanan dengan memanggil method-method spesifik: savePlayers() menyimpan data pemain, saveTurnState() menyimpan status giliran, saveProperties() menyimpan kepemilikan dan kondisi properti, saveDecks() menyimpan kartu yang tersisa di dek, dan saveLogs() menyimpan riwayat aksi. Method loadGame() melakukan proses sebaliknya secara simetris, memulihkan state dari file yang ada.

SaveLoadManager berinteraksi dengan seluruh komponen model seperti Player, Board, TurnManager, CardDeck, dan LogManager karena harus mengakses dan memulihkan setiap bagian state permainan.

2.50. LogManager

Kelas LogManager bertanggung jawab mencatat seluruh aktivitas yang terjadi selama permainan untuk keperluan audit dan tampilan riwayat. Inner class LogEntry mewakili satu entri log dengan atribut turnNumber, username, actionType, dan detail. Atribut logs adalah vektor yang menyimpan seluruh entri log secara kronologis.

Method addLog() mencatat entri baru ke dalam log, clear() menghapus seluruh riwayat (digunakan saat memulai permainan baru), getLogs() mengembalikan seluruh log, dan getRecentLogs() mengembalikan sejumlah log terbaru untuk tampilan ringkas. LogManager dipanggil oleh berbagai manajer dan controller setiap kali aksi signifikan terjadi (pembelian properti, pembayaran sewa, penggunaan kartu, dan sebagainya). Log yang tersimpan juga diserialisasi oleh SaveLoadManager agar riwayat permainan dapat dipulihkan.

2.51. LogManager::LogEntry

Kelas ini merepresentasikan satu baris log. Atributnya menyimpan turn, actor, action, dan detail. Data ini kemudian diubah menjadi LogItem untuk GUI.

2.52. BankruptcyManager

BankruptcyManager akan berinteraksi dengan status game dari Game dan GameContext, dan menginisiasikan fase bangkrut pada player. Player yang tidak memiliki uang yang cukup untuk membayar denda akan dibawa ke fase bankruptcy, dimana seluruh proses bankruptcy (menjual asset ke bank, mortgage asset ke bank, pengecekan uang) akan dihandle olh BankruptcyManager. Jika player tidak dapat membayar denda dengan asset dan uangnya, maka bankruptcy akan menghandle deklarasi player menjadi bangkrut.

2.53. BankruptcyManager::DebtSession

Kelas ini menyimpan satu sesi hutang: debtor, creditor, amount, dan flag debt to bank. Dipakai BankruptcyManager untuk mengantri beberapa proses hutang.

2.54. FestivalManager

FestivalManager akan berinteraksi dengan status game dan menghandle aksi festival yang terjadi di game. FestivalManager digunakan untuk mengaktifkan festival pada tile tertentu jika diaktifkan, menghandle durasi festival pada tile tersebut, dan juga menghandle aksi penguatan festival pada sebuah tile.

2.55. WinConditionManager

WinConditionManager akan berinteraksi dengan status game untuk menghandle proses berakhirnya game dan penentuan pemenang. WinConditionManager digunakan untuk mengecek apakah sudah mencapai MaxTurn ataupun jika sisa 1 player yang tidak bangkrut, dan melakukan inisiasi gameover serta menentukan pemenang jika kondisi tersebut dipenuhi.

2.56. Tile

Kelas Tile adalah kelas abstrak dasar yang merepresentasikan satu petak di papan permainan. Setiap petak memiliki atribut index (posisi di papan), code (kode unik), dan name (nama tampilan). Method virtual murni onLand() harus diimplementasikan oleh setiap subkelas dan dipanggil saat pemain mendarat di petak tersebut.

2.57. ActionTile

Kelas ActionTile mencakup petak-petak yang memicu aksi khusus seperti CardTile, SpecialTile, TaxTile, dan FestivalTile. Kelas ini menjadi base class untuk tile yang punya efek saat didarati. Keberadaan ActionTile memungkinkan kode pengelola petak (seperti Board) untuk membedakan secara tipe antara petak aksi dan petak yang dapat dimiliki (OwnableTile) tanpa harus memeriksa jenis spesifik setiap petak. ActionTile menjadi induk dari empat kategori turunan: CardTile (petak kartu), SpecialTile (petak khusus), TaxTile (petak pajak), dan FestivalTile (petak festival). Seluruh kelas ini memiliki satu kesamaan: pendaratan di atasnya memicu suatu aksi yang dikelola sepenuhnya oleh controller, bukan oleh pemain.

2.58. OwnableTile

Kelas OwnableTile adalah kelas abstrak turunan dari Tile yang menjadi induk dari semua petak yang dapat dibeli, dimiliki, dan diperdagangkan oleh pemain. Kelas ini mendefinisikan seluruh atribut dan perilaku umum kepemilikan properti. Atribut owner (pointer ke Player) menyimpan referensi ke pemilik petak saat ini, atau nullptr jika belum dimiliki. Atribut ownershipStatus mencatat status kepemilikan secara eksplisit. Atribut purchasePrice (int) adalah harga beli properti, dan mortgageValue (int) adalah nilai yang diterima saat properti digadaikan. Atribut festivalMultiplier (int, default 1) dan festivalDuration (int, default 0) menyimpan efek festival yang sedang aktif pada properti ini.

2.59. StreetTile

Kelas ini merupakan properti jalan dengan color group dan bangunan. Methodnya menghitung rent berdasarkan level bangunan, membangun/menjual bangunan, dan mengecek hotel. Kelas ini dipakai PropertyManager, GUITileController, dan Board untuk aturan monopoli warna. StreetTile berelasi dengan RentContext untuk mendapatkan informasi monopoli warna, dengan Player sebagai pemilik, dan dengan PropertyManager yang mengatur validasi pembangunan (memastikan bangunan merata dalam satu kelompok warna) sebelum memanggil buildHouse() atau buildHotel().

2.60. RailroadTile

Kelas RailroadTile adalah kelas konkret turunan dari OwnableTile yang merepresentasikan petak stasiun kereta api. Kelas ini tidak menambahkan atribut baru seluruh datanya berasal dari OwnableTile. Terdapat empat RailroadTile dalam satu permainan.

Method calculateRent(const RentContext& rentContext) mengimplementasikan sewa stasiun: jika tidak ada pemilik atau properti stergadai, sewa adalah 0. Sewa aktual diambil langsung dari rentContext.getRailroadRent() yang sudah dihitung oleh PropertyManager::createRentContext() berdasarkan jumlah stasiun yang dimiliki oleh pemilik yang sama, kemudian dikalikan dengan getFestivalMultiplier(). Sistem ini memungkinkan sewa meningkat secara progresif semakin banyak stasiun yang dimiliki satu pemain, tanpa RailroadTile perlu mengakses Board secara langsung.

2.61. UtilityTile

Kelas ini merupakan properti utilitas. Rent dihitung berdasarkan hasil dadu dan jumlah utility yang dimiliki. Class ini berelasi dengan Dice, RentContext, dan PropertyManager.

Method calculateRent(const RentContext& rentContext) mengimplementasikan mekanisme sewa yang unik: jika tidak ada pemilik atau properti tergadai, sewa adalah 0. Nilai diceTotal adalah total lemparan dadu pemain yang mendarat, dan utilityMultiplier adalah pengali yang ditentukan oleh jumlah utilitas yang dimiliki pemilik (satu utilitas = pengali lebih kecil, dua utilitas = pengali lebih besar) yang sudah disiapkan oleh PropertyManager::createRentContext(). Ini berarti sewa bersifat dinamis dan bergantung pada keberuntungan dadu, berbeda dengan jenis properti lainnya.

2.62. TaxTile

Kelas ini merupakan base class untuk tile pajak. Atributnya menyimpan nilai pajak dan menyediakan perilaku umum untuk turunannya yaitu IncomeTaxTile dan LuxuryTaxTile. TaxTile tetap abstrak karena cara perhitungan pajak berbeda antara IncomeTaxTile dan LuxuryTaxTile.

Dengan mendefinisikan taxAmount di level TaxTile, kedua subkelas berbagi struktur data dasar yang sama, sementara logika perhitungan spesifik didelegasikan ke masing-masing subkelas. TaxManager berinteraksi dengan subkelas konkret untuk memproses pembayaran pajak kepada bank.

2.63. IncomeTaxTile

Kelas IncomeTaxTile merepresentasikan petak Pajak Penghasilan yang memberikan pemain dua opsi pembayaran. Kelas ini mewarisi taxAmount dari TaxTile dan menambahkan dua atribut: flatTax (int) sebagai jumlah pajak tetap yang dapat dipilih pemain, dan percentageTax (int) sebagai persentase dari total kekayaan pemain sebagai alternatif pembayaran.

Method calculateFlatTax() mengembalikan nilai flatTax secara langsung. Method calculatePercentageTax menghitung persentase pajak, di mana playerWealth adalah total kekayaan pemain yang dihitung oleh TaxManager::calculateTotalWealth() (mencakup uang tunai, nilai properti, dan nilai bangunan). Method getFlatTax() dan getPercentageTax() menyediakan akses ke parameter mentah.

2.64. LuxuryTaxTile

Kelas LuxuryTaxTile merepresentasikan tile pajak mewah yang memberikan efek pajak tetap tanpa alternatif. Kelas ini mewarisi taxAmount dari TaxTile dan menambahkan method calculateLuxuryTax yang mengembalikan nilai taxAmount secara langsung.

Method onLand() mengembalikan TileType::LuxuryTax. Berbeda dengan IncomeTaxTile, tidak ada pilihan bagi pemain — jumlah yang dibayar selalu sama persis dengan taxAmount. TaxManager memanggil calculateLuxuryTax() untuk mendapatkan jumlah yang harus dipungut dan memindahkan uang dari pemain ke bank. Kesederhanaan kelas ini mencerminkan aturan permainan yang juga sederhana untuk petak ini.

2.65. CardTile

Kelas CardTile adalah kelas abstrak turunan dari ActionTile yang merepresentasikan petak-petak yang mengharuskan pemain menarik kartu saat mendarat di tile tersebut. Kelas ini menambahkan satu atribut: deckType (string) yang mengidentifikasi jenis dek kartu yang terkait dengan petak ini.

Method getDeckType() menyediakan akses ke nilai deckType tersebut. Atribut ini digunakan oleh controller untuk menentukan dek mana yang harus dikocok — apakah dek Kesempatan atau dek Kas Umum — melalui CardManager. CardTile tetap abstrak karena onLand() belum diimplementasikan, menyerahkan implementasi konkret ke subkelas ChanceTile dan CommunityChestTile. Dengan memisahkan konsep

"petak yang menarik kartu" ke kelas tersendiri, kode controller tidak perlu tahu secara spesifik jenis kartu apa yang ditarik namun cukup memanggil method yang sesuai pada CardManager berdasarkan deckType.

2.66. ChanceTile

Kelas ChanceTile adalah kelas konkret turunan dari CardTile yang merepresentasikan petak Kesempatan pada papan Nimonspoli. Kelas ini tidak menambahkan atribut baru selain yang didefinisikan CardTile.

Method onLand() mengembalikan TileType::Chance, menandakan kepada controller bahwa pemain yang mendarat di petak ini harus menarik kartu dari dek Kesempatan. Controller kemudian memanggil CardManager::drawChanceCard() dan langsung mengeksekusi efek kartu tersebut terhadap pemain melalui method apply(). Dalam papan Monopoli standar, terdapat tiga petak ChanceTile yang tersebar di posisi strategis. Kelas ini bersifat sederhana karena seluruh logika penanganan kartu didelegasikan ke CardController dan CardManager, sesuai prinsip pemisahan tanggung jawab.

2.67. CommunityChestTile

Kelas CommunityChestTile adalah kelas konkret turunan dari CardTile yang merepresentasikan petak Dana umum pada Nimonspoli. Struktur dan perilakunya sejajar dengan ChanceTile, dengan perbedaan pada jenis dek yang dipicu. Method onLand() mengembalikan TileType menandakan pemain harus mengambil kartu dari tile Community Chest ini melalui method drawCommunityChestCard. Kartu Kas Umum umumnya berhubungan dengan transaksi keuangan langsung antar pemain atau antara pemain dan bank (seperti BirthdayCard, DoctorFeeCard).

2.68. FestivalTile

Kelas FestivalTile merepresentasikan petak festival yang berfungsi untuk memberikan player kesempatan untuk mengaktifkan status festival pada salah satu properti yang dimiliki oleh player. Tile dapat mengembalikan identifier dirinya lewat method onLand(), dimana controller dapat mengenali tile dan memproses aktivasi festival lewat FestivalManager jika player mengunjungi tile tersebut.

2.69. SpecialTile

Kelas SpecialTile adalah kelas abstrak turunan dari ActionTile yang menjadi induk dari petak-petak dengan fungsi khusus yang tidak termasuk dalam kategori kartu atau pajak. Kelas ini tidak menambahkan atribut atau method baru, sehingga perannya murni sebagai pengelompokan tipe untuk petak-petak istimewa: GoTile, JailTile, GoToJailTile, dan FreeParkingTile.

Keberadaan SpecialTile memungkinkan pengecekan tipe yang bersih jika diperlukan, dan mengelompokkan secara logis petak-petak yang memiliki aturan unik dalam permainan Monopoli. Setiap subkelas konkretnya mengimplementasikan perilaku yang sangat berbeda satu sama lain, sehingga abstraksi pada level SpecialTile tidak memaksakan atribut atau logika bersama yang mungkin tidak relevan untuk semua anggotanya.

2.70. GoTile

Kelas GoTile merepresentasikan petak start dari board sekaligus berfungsi untuk memberikan salary pada player jika melewati petak tersebut lewat pergerakan langsung tanpa aksi lainnya (teleportasi, masuk ke penjara). Tile dapat mengembalikan identifier dirinya lewat method onLand(), dimana controller dapat mengenali tile ketika player mengunjungi GoTile dan melanjutkan proses pemberian salary. Tile ini juga memiliki method getSalary() untuk mengembalikan jumlah uang salary yang akan diberikan pada player.

2.71. JailTile

Kelas JailTile merepresentasikan petak Jail pada board yang berfungsi sebagai petak peletakkan player ketika dibawa ke penjara. Tile ini dapat mengembalikan identifier dirinya lewat method onLand(), dimana controller dapat mengenali bahwa player mengunjungi penjara, dan akan mengembalikan proses penjara jika player berstatus JAILED dengan JailManager.

2.72. GoToJailTile

Kelas GoToJailTile merepresentasikan petak GoToJail pada board game yang akan membawa player ke dalam jail jika dikunjungi. Tile ini dapat mengembalikan identifier dirinya lewat method onLand(), dimana controller dapat mengenal tile lewat method tersebut dan membawa player ke penjara dengan JailManager jika dikunjungi.

2.73. FreeParkingTile

Kelas FreeParkingTile merepresentasikan petak Free Parking yang tidak memiliki efek tambahan pada sebuah player jika dikunjungi. Dikarenakan tidak ada efek tambahan, maka tile tidak memiliki atribut atau method khusus dan hanya memiliki method untuk mengeluarkan identifier TileType FreeParking

2.74. Card

Kelas ini merupakan base class bagi segala jenis kartu. Kelas ini merupakan abstract class yang menyediakan fungsi virtual apply untuk diimplementasikan oleh inheritornya. Kelas ini menyimpan atribut nama dan deskripsi kartu.

2.75. InstantCard

Kelas ini merupakan base class bagi kartu instant. Kelas ini merupakan inheritor dari Card. Kelas ini merupakan abstract class yang menyediakan fungsi virtual apply untuk diimplementasikan oleh inheritornya.

2.76. HandCard

Kelas ini merupakan base class bagi kartu hand / tangan. Kelas ini merupakan inheritor dari Card. Kelas ini merupakan abstract class yang menyediakan fungsi virtual apply untuk diimplementasikan oleh inheritornya.

2.77. ChanceCard

Kelas ini merupakan base class bagi kartu chance / kesempatan. Kelas ini merupakan inheritor dari InstantCard. Kelas ini merupakan abstract class yang menyediakan fungsi virtual apply untuk diimplementasikan oleh inheritornya.

2.78. CommunityChestCard

Kelas ini merupakan base class bagi kartu community chest / dana umum. Kelas ini merupakan inheritor dari InstantCard. Kelas ini merupakan abstract class yang menyediakan fungsi virtual apply untuk diimplementasikan oleh inheritornya.

2.79. BirthdayCard

Kelas ini merupakan kelas yang bertanggung jawab untuk menyimpan data dan implementasi bagi Kartu Birthday. Kelas ini merupakan inheritor dari CommunityChestCard. Kelas ini menyimpan method polimorfisme apply yang menambahkan uang milik player menggunakan Player, Game, dan BankruptcyManager.

2.80. CampaignFeeCard

Kelas ini merupakan kelas yang bertanggung jawab untuk menyimpan data dan implementasi bagi Kartu Campaign Fee. Kelas ini merupakan inheritor dari CommunityChestCard. Kelas ini menyimpan method polimorfisme apply yang mengurangi uang milik player menggunakan Player, Game, dan BankruptcyManager.

2.81. DoctorFeeCard

Kelas ini merupakan kelas yang bertanggung jawab untuk menyimpan data dan implementasi bagi Kartu Doctor Fee. Kelas ini merupakan inheritor dari CommunityChestCard. Kelas ini menyimpan method polimorfisme apply yang mengurangi uang milik player menggunakan Player, Game, dan BankruptcyManager.

2.82. GoToJailCard

Kelas ini merupakan kelas yang bertanggung jawab untuk menyimpan data dan implementasi bagi Kartu Go To Jail. Kelas ini merupakan inheritor dari ChanceCard. Kelas ini menyimpan method polimorfisme apply yang memindahkan player ke penjara menggunakan JailManager dan Board.

2.83. GoToNearestRailroadCard

Kelas ini merupakan kelas yang bertanggung jawab untuk menyimpan data dan implementasi bagi Kartu Go To Nearest Railroad. Kelas ini merupakan inheritor dari ChanceCard. Kelas ini menyimpan method polimorfisme apply yang memindahkan player ke railroad terdekat menggunakan MovementManager dan Board.

2.84. MoveBackThreeCard

Kelas ini merupakan kelas yang bertanggung jawab untuk menyimpan data dan implementasi bagi Kartu Move Back Three. Kelas ini merupakan inheritor dari ChanceCard. Kelas ini menyimpan method polimorfisme apply yang memundurkan player sejumlah 3 langkah.

2.85. MoveCard

Kelas ini merupakan kelas yang bertanggung jawab untuk menyimpan data dan implementasi bagi Kartu Move. Kelas ini merupakan inheritor dari HandCard. Kelas ini menyimpan atribut steps serta method polimorfisme apply yang memindahkan player sejumlah langkah yang telah ditentukan menggunakan MovementManager.

2.86. TeleportCard

Kelas ini merupakan kelas yang bertanggung jawab untuk menyimpan data dan implementasi bagi Kartu Teleport. Kelas ini merupakan inheritor dari HandCard. Kelas ini menyimpan atribut targetTileIndex serta method polimorfisme apply yang memindahkan player melalui MoevementManager.

2.87. DemolitionCard

Kelas ini merupakan kelas yang bertanggung jawab untuk menyimpan data dan implementasi bagi Kartu Demolition. Kelas ini merupakan inheritor dari HandCard. Kelas ini menyimpan atribut targetStreet serta method polimorfisme apply yang menargetkan street tile lalu menghancurkan bangunannya.

2.88. DiscountCard

Kelas ini merupakan kelas yang bertanggung jawab untuk menyimpan data dan implementasi bagi Kartu Discount. Kelas ini merupakan inheritor dari HandCard. Kelas ini menyimpan atribut discoutPercent dan duration serta method polimorfisme apply yang mengubah status diskon di player.

2.89. ShieldCard

Kelas ini merupakan kelas yang bertanggung jawab untuk menyimpan data dan implementasi bagi Kartu Shield. Kelas ini merupakan inheritor dari HandCard. Kelas ini menyimpan atribut duration serta method polimorfisme apply yang menyalakan shield pada player.

2.90. LassoCard

Kelas ini merupakan kelas yang bertanggung jawab untuk menyimpan data dan implementasi bagi Kartu Lasso. Kelas ini merupakan inheritor dari HandCard. Kelas ini menyimpan atribut targetPlayer yang diatur oleh controller serta method polimorfisme apply yang memindahkan target dan mencatat log.

2.91. NimonException

Kelas ini merupakan kelas yang bertanggung jawab sebagai parent class untuk kelas exception lainnya. Kelas ini merupakan inheritor dari std::exception. Kelas exception lainnya merupakan inheritor dari kelas ini. Kelas ini menyimpan atribut pesan error dan method override what sehingga kelas turunannya dapat memberikan pesan error yang lebih spesifik sesuai dengan jenis exceptionnya.

2.92. InsufficientFundsException

Kelas ini merupakan kelas yang bertanggung jawab untuk menyediakan exception saat terjadi kondisi uang pemain kurang, seperti tidak sanggup membayar uang beli properti. Kelas ini hanya menyediakan constructor yang akan di-throw oleh kelas lain, seperti PropertyManager, ketika mendeteksi kekurangan uang player.

2.93. ForcedInsufficientFundsException

Kelas ini merupakan kelas yang bertanggung jawab untuk menyediakan exception saat terjadi kekurangan uang pada kondisi pembayaran wajib, seperti denda penjara atau pajak. Kelas ini hanya menyediakan constructor yang akan di-throw oleh kelas lain, seperti JailManager dan TaxManager, ketika mendeteksi kekurangan uang player.

2.94. CardSlotFullException

Kelas ini merupakan kelas yang bertanggung jawab untuk menyediakan exception saat slot kartu tangan penuh. Kelas ini hanya menyediakan constructor yang akan di-throw oleh kelas lain, seperti CardManager, ketika mendeteksi aksi yang tidak valid saat slot kartu tangan penuh.

2.95. InvalidActionException

Kelas ini merupakan kelas yang bertanggung jawab untuk menyediakan exception saat terjadi aksi yang tidak valid pada state permainan saat ini. Kelas ini hanya menyediakan constructor yang akan di-throw oleh kelas lain, seperti Dice dan Game, ketika mendeteksi aksi yang tidak valid.

2.96. InvalidTileException

Kelas ini merupakan kelas yang bertanggung jawab untuk menyediakan exception saat terjadi tile index/kode/aksi yang tidak valid. Kelas ini hanya menyediakan constructor yang akan di-throw oleh kelas lain, seperti TileManager, ketika mendeteksi aksi yang tidak valid.

2.97. FileException

Kelas ini merupakan kelas yang bertanggung jawab untuk menyediakan exception saat terjadi kegagalan baca/tulis file. Kelas ini hanya menyediakan constructor yang akan di-throw oleh kelas lain, seperti ConfigManager dan SaveLoadManager, ketika mereka mendeteksi kegagalan.

3. Justifikasi Desain

Kelas dibagi menjadi objek yang bertanggung jawab untuk memenuhi aspek game nimonspoli (Card, Tile, Board), dan objek manager yang bertanggung jawab untuk menghandle keseluruhan objek aspek game tersebut dan menghandle keberjalanan game. Manager tersebut merupakan method class yang akan mengubah isi data objek untuk melaksanakan kuberjalan game, dan setiap manager selalu dibagi menjadi satu tanggung jawab saja (TurnManager, MoveManager, PropertyManager, dll.). Sebuah class Game didesain sebagai komposisi dari seluruh manager tersebut untuk menambahkan enkapsulasi sekaligus menambahkan dependency injection. Sebuah class GameContext yang akan memegang seluruh atribut penting dari Game ditambahkan untuk memegang konteks/state pada game saat itu juga. Manager akan berinteraksi dengan GameContext untuk mengubah state dari game, dan akan memanggil Game jika membutuhkan method tambahan dari manager lainnya.

Selain itu, desain kelas juga menerapkan konsep inheritance pada objek aspek game seperti Tile dan Card. Sebuah abstract base class di desain dan diimplementasikan sebagai blueprint awal, dimana class tersebut akan diturunkan menjadi kelompok-kelompok class yang memiliki fungsional yang lebih spesifik. Adanya abstract base class dan inheritance yang didesain membantu membuka jalan untuk polymorphism yang akan menambahkan abstraksi pada method di Manager sekaligus menghindari konsep prosedural saat implementasi.

Inheritance pada class Tile dan Card juga membantu dalam mengimplementasi sebuah blueprint standard pada masing-masing turunan class tersebut sehingga menghindari adanya class turunan yang tidak memiliki atribut ataupun method yang integral dan diimplementasikan oleh parent classnya. Sebuah class exception juga didesain dan diimplementasikan di dalam program untuk menghandle error yang terjadi selama game. Class exception tersebut akan diturunkan menjadi class exception yang akan menghandle kasus spesifik di dalam game

Pada desain kelas di dalam laporan ini, terdapat banyak class yang mengalami perubahan dibandingkan desain kelas dari milestone 1. Beberapa class ditambahkan untuk menghandle tanggung jawab yang baru (seperti class ForcedInsufficientFundsException untuk menghandle uang tidak cukup tetapi wajib membayar, UIToolkit untuk menghandle GUI dari Nimonspoli, dan GameContext untuk menyimpan state dan context dari game). Penambahan class ini diimplementasikan karena terdapat beberapa desain class pada milestone 1 yang memiliki tanggung jawab berlebih, dapat menyebabkan terjadinya circular dependency, ataupun terdapat fitur dan tanggung jawab yang bertambah selama pengembangan program. Berbagai method di dalam class juga mengalami perubahan ataupun penghapusan, karena terdapat beberapa method yang tidak memiliki fungsional yang jelas pada program, ataupun terdapat method yang melakukan perubahan yang berlebihan daripada tanggung jawab seharusnya. Perubahan method ini mayoritas terjadi pada class Manager yang terdapat di dalam program, dan method diubah agar menyesuaikan dengan tanggung jawab dari class sekaligus menyesuaikan dengan perubahan class lainnya (seperti penambahan GameContext). Secara keseluruhan, perubahan desain class dilakukan untuk menjaga integritas dari atribut dan method sesuai dengan tanggung jawab masing-masing class, menghindari error tambahan dan circular dependency, dan menambahkan level abstraksi dan enkapsulasi.

Walaupun desain kelas sudah diperbaharui, desain kelas yang digunakan masih terdapat beberapa kelemahan. Salah satu kelemahannya adalah adanya beberapa bagian yang masih bersifat prosedural. Salah satu contohnya adalah implementasi tile. Memang, tile sudah menggunakan polimorfisme melalui fungsi virtual apply, akan tetapi, implementasi sifatnya berada di TileController dan cara menghubungkannya adalah dengan menggunakan identifier sehingga TileController terpaksa melakukan switch untuk membedakan tilenya. Hal tersebut merupakan salah satu kelemahan desain karena apabila polimorfisme dilakukan dengan baik, seharusnya tidak ada lagi implementasi switch. Selain itu, prinsip Open/Closed tidak terlalu kuat karena penambahan tile, card, overlay, atau jenis properti baru masih harus mengubah banyak class. Terdapat juga kebocoran enkapsulasi melewati getter dan setter mutable, seperti Board&. Terdapat juga raw pointer pada beberapa class, hal ini menyebabkan ownership dan lifetime tidak eksplisit. UI dan Controller juga masih lumayan tercampur karena terdapat beberapa fungsi controller yang langsung melakukan output, walaupun melalui UIManager.

Terdapat beberapa kendala yang dialami dalam pemilihan desain OOP. Salah satunya adalah kendala dalam menentukan layering kelasnya dengan tetap mempertahankan sifat OOP. Contohnya adalah saat menentukan layering kelas Tile dan Card menjadi layer Model, View, dan Controller.

4. Penerapan Konsep OOP

4.1. Inheritance & Polymorphism

```
     class Card {
     protected:
         string name;
         string description;

     public:
         Card();
         Card(const string& name, const string& description);
         Card(const Card& other);
         virtual ~Card();
         Card& operator=(const Card& other);

         virtual void apply(Game& game, Player& player) = 0;
         string getName() const;
         string getDescription() const;
};

class InstantCard : public Card {
     public:
         InstantCard();
         InstantCard(const string& name, const string& description);
```

```
        InstantCard(const InstantCard& other);
        virtual ~InstantCard();
        InstantCard& operator=(const InstantCard& other);

        virtual void apply(Game& game, Player& player) override = 0;
};

    class HandCard : public Card {
    protected:
        bool used;

    public:
        HandCard();
        HandCard(const string& name, const string& description, bool used);
        HandCard(const HandCard& other);
        virtual ~HandCard();
        HandCard& operator=(const HandCard& other);

        virtual void apply(Game& game, Player& player) override = 0;
};

class ChanceCard : public InstantCard {
    public:
        ChanceCard();
        ChanceCard(const string& name, const string& description);
        ChanceCard(const ChanceCard& other);
        virtual ~ChanceCard();
        ChanceCard& operator=(const ChanceCard& other);

        virtual void apply(Game& game, Player& player) override = 0;
};
```

```
class MoveCard : public HandCard {
     private:
         int steps;

     public:
         MoveCard();
         MoveCard(const string& name, const string& description, bool used, int steps);
         MoveCard(const MoveCard& other);
         ~MoveCard();
         MoveCard& operator=(const MoveCard& other);

         void apply(Game& game, Player& player) override;
         int getSteps() const;
};
```

Konsep Inheritance digunakan pada kelompok Class Card, dimana Inheritance digunakan untuk mengelompokkan kartu menjadi beberapa jenis kartu. Adanya inheritance membantu memberikan beberapa atribut yang standar pada kartu (nama, deskripsi) tetapi juga menambahkan atribut spesifik pada kelas jenis kartu tersebut (steps pada MoveCard, dll.). Inheritance juga membantu penggunaan konsep virtual dan override (seperti pada method apply()) dan membantu kita dalam menggunakan polymorphism. Sebagai contoh, konsep inheritance dan polymorphism pada kartu membantu dalam penggunaan sifat kartu kemampuan tanpa harus membuat kasus if else pada setiap jenis kartu kemampuan :

```
     void CardManager::useHandCard(Game& game, Player& player, int cardIndex) {
         if (player.hasUsedHandCardThisTurn()) {
             cout << "Kamu sudah menggunakan kartu kemampuan pada giliran ini!\n"
                  << "Penggunaan kartu dibatasi maksimal 1 kali dalam 1 giliran." << endl;
```

```
            return;
        }
        auto hand = getHandCards(player);
        if (cardIndex < 0 || cardIndex >= (int)hand.size()) {
            cout << "Pilihan tidak valid." << endl;
            return;
        }
        shared_ptr<HandCard> card = hand[cardIndex];
        card->apply(game, player);
        discardHandCard(card);
        player.removeHandCard(cardIndex);
        player.setUsedHandCardThisTurn(true);
}
```

Konsep Inheritance juga digunakan pada Tile dari Board, dimana file Tile.hpp akan berisi atribut dan method yang akan di-inherit dan dimiliki oleh semua anak kelompok jenis tile sehingga tile memiliki beberapa atribut dan method yang standar namun juga memiliki beberapa atribut dan method yang terkhusus pada jenis tile tersebut:

```
    class Tile {
    public:
        enum class TileType {
            Festival = 0,
            Street,
            Railroad,
            Utility,
            IncomeTax,
            LuxuryTax,
            Chance,
```

```
            CommunityChest,
            Go,
            Jail,
            GoToJail,
            FreeParking
        };

    protected:
        int index;
        string code;
        string name;

    public:
        Tile();
        Tile(int index, const string& code, const string& name);
        Tile(const Tile& other);
        virtual ~Tile();
        Tile& operator=(const Tile& other);

        virtual TileType onLand() const = 0;

        int getIndex() const;
        string getCode() const;
        string getName() const;
};

class OwnableTile : public Tile {
    protected:
        Player* owner;
        OwnershipStatus ownershipStatus;
        int purchasePrice;
```

```
        int mortgageValue;

    public:
        OwnableTile();
        OwnableTile(
            int index,
            const string& code,
            const string& name,
            Player* owner,
            OwnershipStatus ownershipStatus,
            int purchasePrice,
            int mortgageValue
        );
        OwnableTile(const OwnableTile& other);
        virtual ~OwnableTile();
        OwnableTile& operator=(const OwnableTile& other);

        virtual TileType onLand() const override = 0;
        Player* getOwner() const;
        void setOwner(Player* owner);
        void setOwnershipStatus(OwnershipStatus ownershipStatus);
        bool isOwned() const;
        bool isMortgaged() const;
        void mortgage();
        void redeem();
        virtual int calculateRent(const RentContext& rentContext) const = 0;
        virtual void acquire(Player& player) = 0;

        OwnershipStatus getOwnershipStatus() const;
        int getPurchasePrice() const;
        int getMortgageValue() const;
```

```
        virtual int getBuildingValue() const = 0;
        virtual int getSelltoBankValue() const = 0;
};

class ActionTile : public Tile {
    public:
        ActionTile();
        ActionTile(int index, const string& code, const string& name);
        ActionTile(const ActionTile& other);
        virtual ~ActionTile();
        ActionTile& operator=(const ActionTile& other);

        virtual TileType onLand() const override = 0;
};

class CardTile : public ActionTile {
    protected:
        string deckType;

    public:
        CardTile();
        CardTile(int index, const string& code, const string& name, const string& deckType);
        CardTile(const CardTile& other);
        ~CardTile();
        CardTile& operator=(const CardTile& other);

        virtual TileType onLand() const override = 0;

        const string& getDeckType() const;
};
```

Tile dibedakan menjadi Tile yang dapat dimiliki (OwnedTile) dan Tile yang memberikan action (ActionTile), dan setiap tile ini dibedakan kembali menjadi beberapa tile yang lebih spesifik (RailRoadTile, FestivalTile, dll.). Penggunaan Inheritence membantu standardisasi dari beberapa atribut dan method dari Tile, sekaligus membuka jalan bagi penggunakaan polymorphism seperti pada method onLand() yang akan mengembalikan tipe dari masing-masing tile tanpa menspesifikasikan dari class masing-masing tile:

```
     void TileController::resolveLanding(Tile& tile, Player& player) {
         switch (tile.onLand()) {
             case Tile::TileType::Street:
                 handleStreetPurchase(static_cast<StreetTile&>(tile));
                 break;
             case Tile::TileType::Railroad:
             case Tile::TileType::Utility: {
                 OwnableTile& property = static_cast<OwnableTile&>(tile);
                 Player* owner = property.getOwner();

                                 if  (owner  ==  nullptr  ||  property.getOwnershipStatus()  ==
OwnershipStatus::BANK) {
                     const int price = discountedPurchasePrice(player, property);
                     if (player.getMoney() < price) {
                                 uiManager.printError("Uang kamu tidak cukup untuk membeli " +
tile.getName() + ".");
                         return;
                 }
```

4.2. Method/Operator Overloading

```
     CardTile& CardTile::operator=(const CardTile& other) {
         if (this != &other) {
```

```
            ActionTile::operator=(other);
            this->deckType = other.deckType;
        }
        return *this;
    }
    ChanceTile& ChanceTile::operator=(const ChanceTile& other) {
        if (this != &other) {
            CardTile::operator=(other);
        }
        return *this;
    }
    DiscountCard& DiscountCard::operator=(const DiscountCard& other) {
        if (this != &other) {
            HandCard::operator=(other);
            discountPercent = other.discountPercent;
            duration = other.duration;
        }
        return *this;
    }
    Player& Player::operator+=(int amount) {
        this->money += amount;
        return *this;
    }

    Player& Player::operator-=(int amount) {
        this->money -= amount;
        return *this;
    }

    Player Player::operator+(int amount) const {
        Player result = *this;
```

```
        result.money += amount;
        return result;
    }

    Player Player::operator-(int amount) const {
        Player result = *this;
        result.money -= amount;
        return result;
    }
```

Hampir seluruh kelas mengimplementasikan operator overloading pada operator= untuk membuat sebuah copy constructor pada objek tersebut. Tidak hanya itu, kelas Player juga mengimplementasikan operator overloading tambahan pada operator lainnya (+, -, +=, -=) sebagai method yang akan menambah dan mengurangi uang secara langsung. Implementasi operator overloading ini dilakukan untuk mempermudah pengerjaan dan menambah sebuah copy constructor pada mayoritas kelas. Operator overloading pada player juga diimplementasikan agar kelas Manager yang menggunakan player dan uang dapat mengakses data uang player dan menambah/mengurangi player secara cepat dan intuitif tanpa harus menggunakan sebuah method lainnya.

```
    BirthdayCard::BirthdayCard()
        : CommunityChestCard(), amount(0) {}

    BirthdayCard::BirthdayCard(const string& name, const string& description, int amount)
        : CommunityChestCard(name, description), amount(amount) {}

    BirthdayCard::BirthdayCard(const BirthdayCard& other)
   : CommunityChestCard(other), amount(other.amount) {}
    AuctionManager::AuctionManager(
        int currentBid,
        Player* highestBidder,
```

```
        const vector<Player*>& participants,
        int currentAuctionPlayerIndex,
        int consecutivePasses,
        bool auctionActive,
        bool hasAnyBid,
        shared_ptr<StreetTile> currentTile,
        Player* triggerPlayer
    ) : currentBid{currentBid},
        highestBidder{highestBidder},
        participants{participants},
        currentAuctionPlayerIndex{currentAuctionPlayerIndex},
        consecutivePasses{consecutivePasses},
        auctionActive{auctionActive},
        hasAnyBid{hasAnyBid},
        currentTile{currentTile},
        triggerPlayer{triggerPlayer}
        {}

    AuctionManager::AuctionManager(const AuctionManager& other) {
        if (this != &other) {
            this->currentBid = other.currentBid;
            this->highestBidder = other.highestBidder;
            this->participants = other.participants;
            this->currentAuctionPlayerIndex = other.currentAuctionPlayerIndex;
            this->consecutivePasses = other.consecutivePasses;
            this->auctionActive = other.auctionActive;
            this->hasAnyBid = other.hasAnyBid;
            this->currentTile = other.currentTile;
            this->triggerPlayer = other.triggerPlayer;
        }
}
```

```
    CommunityChestTile::CommunityChestTile()
        : CardTile()
    {}

    CommunityChestTile::CommunityChestTile(int index, const string& code, const string& name,
const string& deckType)
        : CardTile(index, code, name, deckType)
    {}

    CommunityChestTile::CommunityChestTile(const CommunityChestTile& other)
        : CardTile(other)
{}
```

Method overloading juga digunakan pada mayoritas kelas sebagai bentuk lain dari copy constructor yang dapat dilakukan oleh objek. Adanya method overloading pada constructor membantu dalam mengimplementasi copy constructor dan juga mengimplementasi default constructor dari beberapa kelas.

4.3. Template & Generic Classes

```
    template <typename T>
    class CardDeck {
    private:
        vector<shared_ptr<T>> drawPile;
        vector<shared_ptr<T>> discardPile;

    public:
```

```
        CardDeck() = default;
             CardDeck(const  vector<shared_ptr<T>>&  drawPile,  const  vector<shared_ptr<T>>&
discardPile)
            : drawPile(drawPile), discardPile(discardPile) {}
        CardDeck(const CardDeck<T>& other) = default;
        ~CardDeck() = default;
        CardDeck<T>& operator=(const CardDeck<T>& other) = default;

        void addCard(shared_ptr<T> card) {
            drawPile.push_back(card);
        }
        shared_ptr<T> drawCard() {
            if(drawPile.empty()) {
                reshuffleDiscardIntoDrawPile();
            }
            shared_ptr<T> card = drawPile.back();
            drawPile.pop_back();
            return card;
        }
        void discardCard(shared_ptr<T> card) {
            discardPile.push_back(card);
        }

        void shuffle() {
            for (int i = (int)drawPile.size() - 1; i > 0; i--) {
                int j = rand() % (i + 1);
                swap(drawPile[i], drawPile[j]);
            }
        }

        void reshuffleDiscardIntoDrawPile() {
```

```
            for (auto& card : discardPile) {
                drawPile.push_back(card);
            }
            discardPile.clear();
            shuffle();
        }

        bool isEmpty() const {
            return drawPile.empty();
        }

        int size() const {
            return (int)drawPile.size();
        }

        const vector<shared_ptr<T>>& getDrawPile() const { return drawPile; }
        const vector<shared_ptr<T>>& getDiscardPile() const { return discardPile; }

        void clear() {
            drawPile.clear();
            discardPile.clear();
        }
};
```

Template class diimplementasikan di dalam kelas CardDeck. CardDeck merupakan kelas manager yang akan mengatur tumpukan kartu yang ada di dalam board, yaitu tumpukan kartu dana dan tumpukan kartu kesempatan. Template class ini diimplementasikan dikarenakan kedua tumpukan kartu memiliki fungsionalitas yang sama, namun memiliki komponen kartu yang berbeda (Kartu Dana vs Kartu Kesempatan), sehingga fungsionalitas template class digunakan dan diimplementasikan untuk membuat sebuah class manager yang akan menghandle kedua tumpukan tersebut dan memiliki atribut dan method yang sama, namun terdiri dari komponen atribut yang berbeda (Kartu Dana dengan Kartu Kesempatan)

4.4. Exception

```
class NimonException : public std::exception {
    protected:
        std::string message;
    public:
        NimonException(const std::string& message) : message(message){}
        virtual const char* what() const noexcept override {
            return message.c_str();
        }
};

class InsufficientFundsException : public NimonException {
    public:
        InsufficientFundsException(int required, int available)
            : NimonException("Uang tidak cukup. Dibutuhkan: M" + std::to_string(required) + ",
Tersedia: M" + std::to_string(available)) {}
};

class ForcedInsufficientFundsException : public NimonException {
    public:
        ForcedInsufficientFundsException(int required, int available)
            : NimonException("Uang tidak cukup. Dibutuhkan : M" + std::to_string(required) + ",
Tersedia: M" + std::to_string(available) + ". Pengecekkan Bankcruptcy...")){}
};

class CardSlotFullException : public NimonException {
public:
    CardSlotFullException()
        : NimonException("Slot kartu penuh. Maksimal 3 kartu di tangan.") {}
};

class InvalidActionException : public NimonException {
```

```
public:
    explicit InvalidActionException(const std::string& message)
        : NimonException(message) {}
};

class InvalidTileException : public NimonException {
public:
    explicit InvalidTileException(const std::string& code)
        : NimonException("Petak dengan kode '" + code + "' tidak ditemukan.") {}
};

class FileException : public NimonException {
public:
    explicit FileException(const std::string& message)
        : NimonException(message) {}
};
```

Custom exception digunakan karena memberikan struktur penanganan error yang terorganisir, fleksibel, dan mudah dikembangkan. Dengan menurunkan semua exception dari base class NimonException yang meng-extend std::exception, permainan memiliki fleksibilitas untuk menangkap error secara umum menggunakan catch(NimonException&) ketika ingin menangani seluruh error game dalam satu tempat, maupun secara spesifik seperti catch(InsufficientFundsException&) ketika diperlukan penanganan khusus. Hierarki ini juga memisahkan error domain-specific (pelanggaran aturan game yang bisa di-recover) dari error sistem teknis sehingga alur game menjadi lebih robust.

Keuntungan lain terletak pada encapsulation konteks error di dalam constructor. Class seperti InsufficientFundsException cukup menerima data mentah berupa required dan available lalu constructor-nya yang bertugas memformat pesan secara konsisten. Hal ini menerapkan prinsip DRY(Don’t Repeat it Yourself) dan single source of truth(jika format pesan perlu diubah, cukup dimodifikasi di satu tempat). Class deskriptif, seperti CardSlotFullException atau InvalidTileException, berfungsi sebagai dokumentasi dan memberikan type safety serta kejelasan semantik dibandingkan melempar std::runtime_error generik.

Pendekatan ini juga bersifat extensible sesuai Open/Closed Principle. Untuk menambah jenis error baru cukup dengan membuat class turunan baru tanpa perlu mengubah catch block yang sudah ada, sementara override what() dengan noexcept memastikan kompatibilitas penuh dengan ekosistem C++ standar.

4.5. C++ Standard Template Library

```
     #pragma once

     #include <vector>

     #include "model/Board.hpp"
     #include "model/Player.hpp"

     class GameContext {
     private:
         int currentTurn;
         int maxTurn;
         bool isRunning;
         Board board;
    std::vector<Player> players;

     string CommandController::toUpperCopy(string value) const {
         transform(value.begin(), value.end(), value.begin(), [](unsigned char ch) {
             return static_cast<char>(toupper(ch));
         });
         return value;
}

     string CommandController::firstToken(const string& input) const {
         istringstream stream(input);
         string token;
```

```
        stream >> token;
        return toUpperCopy(token);
}

    string CommandController::formatMoney(int amount) const {
        const bool negative = amount < 0;
        long long value = amount;
        if (negative) {
            value *= -1;
        }

        string digits = to_string(value);
        string formatted;
        int count = 0;
        for (int i = static_cast<int>(digits.size()) - 1; i >= 0; i--) {
            if (count == 3) {
                formatted.push_back('.');
                count = 0;
            }
            formatted.push_back(digits[static_cast<size_t>(i)]);
            count++;
        }

        reverse(formatted.begin(), formatted.end());

        string prefix = "M";
        if (negative) prefix += "-";
        return prefix + formatted;
}

    int CommandController::wrappedMove(Game& game, Player& player, int steps) const {
```

```
        Board& board = game.getBoard();
        const int boardSize = board.getBoardSize();
        if (boardSize <= 0) {
            player.moveTo(player.getPosition() + steps);
            return player.getPosition();
        }

        const int oldPosition = normalizedPosition(player.getPosition(), boardSize);
        const int rawPosition = oldPosition + steps;
        const int newPosition = normalizedPosition(rawPosition, boardSize);

        if (rawPosition > boardSize) {
            shared_ptr<Tile> goTile = board.getTile(1);
            GoTile* go = dynamic_cast<GoTile*>(goTile.get());
            int salary = game.getConfigManager().getGoSalary();
            if (go != nullptr) salary = go->getSalary();
            player += salary;
            game.getLogManager().addLog(
                game.getCurrentTurn(),
                player.getUsername(),
                "GO",
                "Melewati GO dan menerima M" + to_string(salary)
            );
        }

        player.moveTo(newPosition);
        return newPosition;
}
```

Cuplikan kode tersebut berisi beberapa aplikasi dari C++ STL yang digunakan. C++ STL yang digunakan dalam program ini secara keseluruhan adalah : <algorithm>, <array>, <cctype>, <cmath>, <cstdlib>, <ctime>, <deque>, <exception>, <fstream>, <functional>, <iomanip>, <iostream>, <istream>, <limits>, <map>, <memory>, <ostream>, <random>, <sstream>, <stdexcept>, <string>, <utility>, dan <vector>. Keseluruhan dari STL tersebut digunakan untuk mempermudah implementasi dari berbagai fungsional kelas dan membantu dalam memanajemen memori dan objek kelas-kelas tersebut. Contohnya, vector digunakan untuk membuat sebuah array dinamis tanpa harus memanajemen memori secara langsung, string digunakan untuk membuat objek berisi string tanpa harus membuat pointer ke char, dan algorithm diimplementasikan untuk menggunakan berbagai fungsi terhadap objek tersebut seperti mereverse string dan membuat semua karakter di string menjadi uppercase. STL tersebut digunakan agar program dapat mencoba untuk memenuhi Rule of 0 sebisa mungkin sebelum terdapat sebuah kelas ataupun metode yang menyebabkan rule of 0 dipenuhi seterusnya.

4.6. Konsep OOP lain

```
#pragma once

#include <string>

using namespace std;

class Game;
class Player;

class Card {
protected:
    string name;
    string description;

public:
    Card();
    Card(const string& name, const string& description);
    Card(const Card& other);
    virtual ~Card();
```

```
        Card& operator=(const Card& other);

        virtual void apply(Game& game, Player& player) = 0;
        string getName() const;
        string getDescription() const;
};

    #pragma once

    #include <string>

    using namespace std;

    class Tile {
    public:
        enum class TileType {
            Festival = 0,
            Street,
            Railroad,
            Utility,
            IncomeTax,
            LuxuryTax,
            Chance,
            CommunityChest,
            Go,
            Jail,
            GoToJail,
            FreeParking
        };

    protected:
```

```
        int index;
        string code;
        string name;

    public:
        Tile();
        Tile(int index, const string& code, const string& name);
        Tile(const Tile& other);
        virtual ~Tile();
        Tile& operator=(const Tile& other);

        virtual TileType onLand() const = 0;

        int getIndex() const;
        string getCode() const;
        string getName() const;
};
```

Konsep Abstract Base Class juga digunakan dalam beberapa kelas, seperti pada kelas Tile dan Card. Abstract class ini digunakan agar terdapat sebuah blueprint untuk kelompok Tile dan Card yang dapat digunakan pada anak class, dimana atribut dan method tersebut menjadi sebuah atribut dan method standar yang wajib diimplementasikan oleh setiap anak class. Adanya abstract base class membantu memperkuat konsep inheritance dari setiap anak class dan membantu standardisasikan dari setiap kelompok class Tile dan Card sekaligus menghindari parent class untuk diinisialisasikan secara tidak sengaja. Beberapa anak class dari Tile dan Card juga diimplementasikan sebagai abstract class lanjutan pada anak dan cucu class, dan digunakan sebagai blueprint standard bagi kelompok class tersebut.

```
    #pragma once

    #include <string>
    #include <vector>
```

```
#include "model/Board.hpp"
#include "model/GameContext.hpp"
#include "model/Dice.hpp"
#include "model/Player.hpp"
#include "model/managers/TurnManager.hpp"
#include "model/managers/MovementManager.hpp"
#include "model/managers/PropertyManager.hpp"
#include "model/managers/AuctionManager.hpp"
#include "model/managers/CardManager.hpp"
#include "model/managers/JailManager.hpp"
#include "model/managers/BankruptcyManager.hpp"
#include "model/managers/SaveLoadManager.hpp"
#include "model/managers/ConfigManager.hpp"
#include "model/managers/LogManager.hpp"
#include "model/managers/TaxManager.hpp"
#include "model/managers/FestivalManager.hpp"
#include "model/managers/WinConditionManager.hpp"

using namespace std;

class Game {
private:

    GameContext gameContext;
    Dice dice;
    TurnManager turnManager;
    MovementManager movementManager;
    PropertyManager propertyManager;
    AuctionManager auctionManager;
    CardManager cardManager;
```

```
    JailManager jailManager;
    BankruptcyManager bankruptcyManager;
    SaveLoadManager saveLoadManager;
    ConfigManager configManager;
    LogManager logManager;
    TaxManager taxManager;
    FestivalManager festivalManager;
    WinConditionManager winConditionManager;
```

Konsep Composition juga digunakan pada kelas Game yang berfungsi sebagai kelas utama yang menyimpan seluruh manager dan konteks game untuk beroperasi. Komposisi ini diimplementasikan untuk membuat sebuah class yang dapat melakukan dependency injection pada GameContext agar kelas lain dapat melakukan operasi dari manager lainnya tanpa harus membuat depedency secara hard-coded.

```
int CommandController::wrappedMove(Game& game, Player& player, int steps) const {
    Board& board = game.getBoard();
    const int boardSize = board.getBoardSize();
    if (boardSize <= 0) {
        player.moveTo(player.getPosition() + steps);
        return player.getPosition();
    }

    const int oldPosition = normalizedPosition(player.getPosition(), boardSize);
    const int rawPosition = oldPosition + steps;
    const int newPosition = normalizedPosition(rawPosition, boardSize);

    if (rawPosition > boardSize) {
        shared_ptr<Tile> goTile = board.getTile(1);
        GoTile* go = dynamic_cast<GoTile*>(goTile.get());
        int salary = game.getConfigManager().getGoSalary();
```

```
            if (go != nullptr) salary = go->getSalary();
            player += salary;
            game.getLogManager().addLog(
                game.getCurrentTurn(),
                player.getUsername(),
                "GO",
                "Melewati GO dan menerima M" + to_string(salary)
            );
        }

        player.moveTo(newPosition);
        return newPosition;
}
```

Adanya kelas Game yang mengimplementasikan composition pada seluruh manager dan GameContext juga membantu dalam mengimplementasikan Dependency Injection pada berbagai kelas, terutama pada kelas controller. Kelas controller tidak perlu melakukan dependency secara hardcoded untuk menggunakan operasi yang ada di dalam kelas-kelas manager, tetapi cukup memanggil game saja untuk mendapatkan manager tertentu, dan melakukan operasi lewat manager yang dipanggil tersebut. Hal ini membantu meningkatkan modularitas dari program, mengurangi kesempatan untuk adanya dependency yang berlebihan serta terjadinya circular dependency, dan membantu meningkatkan encapsulation dan abstraction dari program.

5. Bonus Yang dikerjakan

5.1. Bonus yang diusulkan oleh spek

5.1.1. GUI

GUI Nimonspoli diimplementasikan menggunakan Raylib dengan pengelompokan renderer, controller, dan state tampilan. Hal tersebut dilakukan agar code lebih gampang untuk di-maintain. Secara umum, implementasinya dibagi menjadi:

- RaylibGuiApp
  Menjalankan window, game loop, dan render game setiap

```
while (!WindowShouldClose() && !controller_.state().isQuitRequested()) {
    const float deltaTime = GetFrameTime();

    controller_.tick(deltaTime);
    controller_.handleGlobalShortcuts();

    BeginDrawing();
    ClearBackground(toolkit_.theme().getPaperSoft());

    if (controller_.state().getScreen() == Screen::Gameplay) {
        gameplayRenderer_.draw(controller_, toolkit_);
    } else {
        menuRenderer_.draw(controller_, toolkit_);
    }

    overlayRenderer_.draw(controller_, toolkit_);
    EndDrawing();
}
```

  Kode ini menunjukkan loop utama GUI. Setiap frame, controller di-update lalu renderer menggambar tampilan yang sesuai.

- GUIGameController
  Menjadi penghubung antara backend dengan GUI. Controller ini menyimpan tampilan, melakukan sinkronisasi dengan data backend, dan meneruskan aksi para player

```
void GUIGameController::syncViewFromBackend() {
    GameState view;

    view.setTurn(backendGame_.getCurrentTurn());
    view.setCurrentPlayer(std::max(0, currentBackendPlayerIndex()));
    view.setRolledThisTurn(backendGame_.getTurnManager().isRolledThisTurn());
    view.setLastDieOne(backendGame_.getDice().getDie1());
    view.setLastDieTwo(backendGame_.getDice().getDie2());

    std::vector<TileInfo> board;
    for (const std::shared_ptr<Tile>& tile : backendGame_.getBoard().getTiles()) {
        if (tile != nullptr) {
            board.push_back(makeTileInfoFromBackend(*tile));
        }
    }
    view.setBoard(board);

    appState_.setGame(view);
}
```

Fungsi ini mengubah data backend menjadi data yang siap ditampilkan oleh GUI.

- Renderer
  - MenuScreenRenderer untuk menu utama yang berisi new game, load game, etc

```
void MenuScreenRenderer::draw(GUIGameController& session, const UiToolkit& toolkit) const
{
    switch (session.state().getScreen()) {
        case Screen::MainMenu:
            drawMainMenu(session, toolkit);
            break;
        case Screen::NewGame:
            drawNewGame(session, toolkit);
```

```
            break;
        case Screen::LoadGame:
            drawLoadGame(session, toolkit);
            break;
        case Screen::Tutorial:
            drawTutorial(session, toolkit);
            break;
        default:
            break;
    }
}
```

- GameplayScreenRenderer untuk render board permainan, panel pemain, state permainan, tombol aksi/input player, save game, etc

```
void GameplayScreenRenderer::draw(GUIGameController& session, const UiToolkit& toolkit)
const {
    AppState& state = session.state();

    const Rectangle boardRect = {/* area papan */};
    const Rectangle summaryRect = {/* area info pemain */};
    const Rectangle actionRect = {/* area tombol aksi */};
    const Rectangle logsRect = {/* area log */};

    const bool interactive = !session.isOverlayOpen();

    drawBoardPanel(session, toolkit, boardRect, interactive);
    drawCurrentPlayerPanel(session, toolkit, summaryRect);
    drawActionPanel(session, toolkit, actionRect, interactive);
    drawLogsPreview(session, toolkit, logsRect);
}
```

```
void GameplayScreenRenderer::drawBoardPanel(
    GUIGameController& session,
    const UiToolkit& toolkit,
    Rectangle rect,
    bool interactive
) const {
    AppState& state = session.state();

    for (const TileInfo& tile : state.getGame().getBoard()) {
        const Rectangle tileRect = session.boardTileRect(rect, tile.getIndex());
        const bool hovered = interactive && toolkit.isHovered(tileRect);
        const bool selected = state.getGame().getSelectedTile() == tile.getIndex();

        DrawRectangleRec(
            toolkit.inset(tileRect, 2.0f),
            selected
                ? tile.getAccent()
                : toolkit.mix(toolkit.theme().getPaperSoft(), tile.getAccent(), 0.12f)
        );

        if (interactive && hovered && IsMouseButtonReleased(MOUSE_LEFT_BUTTON)) {
            session.setSelectedTile(tile.getIndex());
        }
    }

    for (const PlayerInfo& player : state.getGame().getPlayers()) {
        const Rectangle tileRect = session.boardTileRect(rect, player.getPosition());

        DrawCircle(
            static_cast<int>(tileRect.x + tileRect.width / 2),
            static_cast<int>(tileRect.y + tileRect.height / 2),
            10.0f,
            player.getAccent()
```

```
        );
    }
}

void GameplayScreenRenderer::drawActionPanel(
    GUIGameController& session,
    const UiToolkit& toolkit,
    Rectangle rect,
    bool interactive
) const {
    const bool canRollDice =
        interactive &&
        session.state().getGame().isTurnStarted() &&
        !session.state().getGame().isRolledThisTurn();

    if (toolkit.drawButton(
            "Roll Dice",
            {/* posisi tombol */},
            toolkit.theme().getCoral(),
            toolkit.theme().getPaperSoft(),
            canRollDice,
            15.0f)) {
        session.rollDice();
    }

    if (toolkit.drawButton(
            "Cards",
            {/* posisi tombol */},
            toolkit.theme().getTeal(),
            toolkit.theme().getInk(),
            interactive,
            15.0f)) {
        session.openCards();
    }
```

```
    if (toolkit.drawButton(
            "Save",
            {/* posisi tombol */},
            toolkit.theme().getGold(),
            toolkit.theme().getInk(),
            interactive,
            15.0f)) {
        session.openSave();
    }
}
```

Kode ini membagi tampilan gameplay menjadi board, panel pemain, tombol aksi, dan log. Board menggambar tile serta posisi pemain. Action panel menangani tombol seperti Roll Dice dan Cards, lalu meneruskan aksinya ke GUIGameController.

- OverlayRenderer untuk popup seperti beli properti, auction, kartu, pajak, jail, save, etc

```
void OverlayRenderer::draw(GUIGameController& session, const UiToolkit& toolkit) const {
    if (!session.isOverlayOpen()) {
        return;
    }

    switch (session.state().getOverlay().getType()) {
        case OverlayType::Purchase:
            drawPurchase(session, toolkit);
            break;
        case OverlayType::Auction:
            drawAuction(session, toolkit);
            break;
        case OverlayType::Cards:
            drawCards(session, toolkit);
            break;
```

```
        case OverlayType::GameOver:
            drawGameOver(session, toolkit);
            break;
        default:
            break;
    }
}
```

- Sub-Controller
  - GUICommandController untuk aksi umum seperti start turn, roll dice, save, shortcut, etc.

```
void GUICommandController::rollDice() {
    if (controller_.appState_.getScreen() != Screen::Gameplay ||
        !controller_.backendGame_.isGameRunning() ||
        controller_.backendGame_.getPlayers().empty()) {
        return;
    }

    if (!controller_.guiTurnStarted_) {
        controller_.addToast("Turn belum siap untuk melempar dadu.", RED);
        return;
    }

    if (controller_.backendGame_.getTurnManager().isRolledThisTurn()) {
        controller_.addToast("Dadu sudah dilempar pada turn ini.", RED);
        return;
    }

    Player& player = controller_.backendGame_.getCurrentPlayer();

    controller_.backendGame_.getDice().roll();
    controller_.backendGame_.getTurnManager().registerDiceResult(
```

```
        controller_.backendGame_.getDice().isDouble()
    );

    controller_.diceRolledThisTurn_ = true;

    const int total = controller_.backendGame_.getDice().getTotal();
    const int destination = controller_.moveBackendPlayer(player, total);

    controller_.addLog(player.getUsername(), "DADU",
        "Lempar " +
        std::to_string(controller_.backendGame_.getDice().getDie1()) +
        "+" +
        std::to_string(controller_.backendGame_.getDice().getDie2()) +
        "=" +
        std::to_string(total)
    );

    controller_.resolveBackendLanding(destination, false);
    finishTurnAfterDiceIfReady();
}
```

Sub-controller ini menangani aksi command utama. Contohnya saat pemain menekan tombol Roll Dice, GUI akan memanggil rollDice(), lalu dice backend dilempar, pemain digerakkan, tile tujuan diproses, dan tampilan diperbarui.

- GUITileController untuk aksi tile seperti beli properti, bayar pajak, auction, jail, mortgage, dan build.

```
void GUITileController::resolveBackendLanding(int backendTileIndex, bool fromMovement) {
    const int normalizedIndex =
        controller_.normalizedBackendTileIndex(backendTileIndex);

    controller_.appState_.getGame().setSelectedTile(
        controller_.uiTileIndexFromBackend(normalizedIndex)
    );
```

```
Tile* tile = controller_.backendGame_.getBoard()
    .getTile(normalizedIndex)
    .get();

Player& player = controller_.backendGame_.getCurrentPlayer();

OverlayType overlayToOpen = OverlayType::None;
int overlayTile = controller_.uiTileIndexFromBackend(normalizedIndex);

switch (tile->onLand()) {
    case Tile::TileType::Street: {
        StreetTile& street = static_cast<StreetTile&>(*tile);
        Player* owner = street.getOwner();

        if (owner == nullptr ||
            street.getOwnershipStatus() == OwnershipStatus::BANK) {
            overlayToOpen = OverlayType::Purchase;
            break;
        }

        if (owner == &player) {
            controller_.addToast("Mendarat di properti sendiri.",
                groupAccent(street.getColorGroup()));
            break;
        }

        // Jika dimiliki pemain lain, controller menghitung dan membayar sewa.
        break;
    }

    case Tile::TileType::Chance:
        controller_.syncViewFromBackend();
        controller_.openRandomCardDraw(kChanceDeckKey);
```

```
            return;

        case Tile::TileType::CommunityChest:
            controller_.syncViewFromBackend();
            controller_.openRandomCardDraw(kCommunityDeckKey);
            return;

        case Tile::TileType::IncomeTax:
            overlayToOpen = OverlayType::IncomeTax;
            break;

        case Tile::TileType::LuxuryTax:
            overlayToOpen = OverlayType::LuxuryTax;
            break;

        case Tile::TileType::Festival:
            overlayToOpen = OverlayType::Festival;
            break;

        default:
            break;
    }

    controller_.syncViewFromBackend();

    if (overlayToOpen == OverlayType::Purchase) {
        openPurchase(overlayTile);
    } else if (overlayToOpen == OverlayType::IncomeTax) {
        openIncomeTax();
    } else if (overlayToOpen == OverlayType::LuxuryTax) {
        openLuxuryTax();
    } else if (overlayToOpen == OverlayType::Festival) {
        openFestival();
    } else {
```

```
        controller_.maybeOpenLiquidation();
        controller_.finishTurnAfterDiceIfReady();
    }
}

void GUITileController::buySelectedProperty() {
    OwnableTile* ownable = controller_.ownableFromUi(
        controller_.appState_.getOverlay().getTileIndex()
    );

    Player& player = controller_.backendGame_.getCurrentPlayer();

    if (ownable == nullptr) {
        controller_.addToast("Petak ini tidak bisa dibeli.", RED);
        return;
    }

    if (ownable->getOwner() != nullptr &&
        ownable->getOwnershipStatus() != OwnershipStatus::BANK) {
        controller_.addToast("Properti sudah dimiliki.", RED);
        return;
    }

    const int price = player.effectiveCost(ownable->getPurchasePrice());

    if (player.getMoney() < price) {
        controller_.addToast("Uang tidak cukup.", RED);
        return;
    }

    player -= ownable->getPurchasePrice();
    ownable->setOwner(&player);
    ownable->setOwnershipStatus(OwnershipStatus::OWNED);
```

```
    controller_.addLog(player.getUsername(), "BELI",
        "Membeli " + ownable->getName() +
        " seharga M" + std::to_string(price) + "."
    );

    controller_.closeOverlay();
    controller_.syncViewFromBackend();
    controller_.finishTurnAfterDiceIfReady();
}
```

Kode ini penting karena menunjukkan cara GUI memproses efek tile setelah pemain bergerak. Jika pemain mendarat di properti kosong, GUI membuka modal pembelian. Jika mendarat di Chance atau Community Chest, GUI membuka modal kartu. Kode ini juga menunjukkan aksi pembelian properti dari GUI. Setelah tombol beli ditekan, controller mengecek validasi, mengurangi uang pemain, mengubah status kepemilikan, menutup overlay, dan menyinkronkan tampilan

- GUICardController untuk aksi kartu.

```
void GUICardController::openRandomCardDraw(int deckKey) {
    clearPendingDrawnCard(true);

    if (deckKey == kCommunityDeckKey) {
        controller_.pendingCommunityChestCard_ =
            controller_.backendGame_.getCardManager().drawCommunityChestCard();

        controller_.appState_.getOverlay().setCard(
            controller_.makeCardInfoFromBackend(
                *controller_.pendingCommunityChestCard_
            )
        );
    } else {
        controller_.pendingChanceCard_ =
```

```
            controller_.backendGame_.getCardManager().drawChanceCard();

        controller_.appState_.getOverlay().setCard(
            controller_.makeCardInfoFromBackend(
                *controller_.pendingChanceCard_
            )
        );
    }

    controller_.appState_.getOverlay().setDeckKey(deckKey);
    controller_.openOverlay(OverlayType::CardDraw);
}

void GUICardController::applyDrawnCard() {
    Player& player = controller_.backendGame_.getCurrentPlayer();
    const int previousPosition = player.getPosition();

    if (controller_.pendingChanceCard_ != nullptr) {
        controller_.pendingChanceCard_->apply(controller_.backendGame_, player);
        controller_.backendGame_.getCardManager()
            .discardChanceCard(controller_.pendingChanceCard_);
        controller_.pendingChanceCard_.reset();
    } else if (controller_.pendingCommunityChestCard_ != nullptr) {
        controller_.pendingCommunityChestCard_->apply(controller_.backendGame_, player);
        controller_.backendGame_.getCardManager()
            .discardCommunityChestCard(controller_.pendingCommunityChestCard_);
        controller_.pendingCommunityChestCard_.reset();
    }

    const int normalizedPosition =
        controller_.normalizedBackendTileIndex(player.getPosition());

    player.moveTo(normalizedPosition);
    controller_.closeOverlay();
```

```
    if (normalizedPosition != controller_.normalizedBackendTileIndex(previousPosition)) {
        controller_.resolveBackendLanding(normalizedPosition, true);
    } else {
        controller_.syncViewFromBackend();
        controller_.maybeOpenLiquidation();
        controller_.finishTurnAfterDiceIfReady();
    }
}
```

Sub-controller ini menangani kartu. Saat pemain mendarat di tile Chance atau Community Chest, controller mengambil kartu dari deck backend, mengubahnya menjadi CardInfo, lalu membuka overlay kartu. Kode ini juga menunjukkan bahwa efek kartu benar-benar diterapkan ke backend. Jika kartu menyebabkan pemain berpindah posisi, tile tujuan akan diproses kembali melalui resolveBackendLanding().

5.1.2. COM

```
#include "model/ComputerDecisionMaker.hpp"
```

```
#include <algorithm>
#include <random>

using namespace std;

static mt19937& rng() {
    static mt19937 gen(random_device{}());
    return gen;
}

int ComputerDecisionMaker::randomInt(int min, int max) {
    if (min > max) return min;
    uniform_int_distribution<int> dist(min, max);
    return dist(rng());
}

bool ComputerDecisionMaker::rollThreshold(int threshold) {
    return randomInt(1, 100) > threshold;
}

string ComputerDecisionMaker::decideCommand() {
    return "LEMPAR_DADU";
}

bool ComputerDecisionMaker::decideToBuy() {
    return rollThreshold(50);
```

```
}

int ComputerDecisionMaker::decideIncomeTax() {
    return randomInt(1, 2);
}

int ComputerDecisionMaker::decideFestival(int maxIndex) {
    if (maxIndex <= 0) return 0;
    return randomInt(1, maxIndex);
}

string ComputerDecisionMaker::decideAuctionAction(int minimumBid, int playerMoney, bool
forcedBid) {
    const int minBid = max(0, minimumBid);
    if (playerMoney < minBid) return "PASS";
    if (!forcedBid && !rollThreshold(25)) return "PASS";
    return "BID " + to_string(randomInt(minBid, playerMoney));
}

int ComputerDecisionMaker::decideLiquidation(int maxIndex) {
    if (maxIndex <= 0) return 0;
    return randomInt(1, maxIndex);
}

int ComputerDecisionMaker::decideForceDropChoice(int maxIndex) {
    if (maxIndex <= 0) return 1;
```

```
    return randomInt(1, maxIndex);
}

int ComputerDecisionMaker::decideAbilityCard(int maxIndex) {
    if (maxIndex <= 0) return 0;
    return randomInt(1, maxIndex);
}

int ComputerDecisionMaker::decideBuildGroup(int maxIndex) {
    if (maxIndex <= 0) return 0;
    return randomInt(1, maxIndex);
}

int ComputerDecisionMaker::decideBuildTile(int maxIndex) {
    if (maxIndex <= 0) return 1;
    return randomInt(1, maxIndex);
}

int ComputerDecisionMaker::decideRedeemChoice(int maxIndex) {
    if (maxIndex <= 0) return 0;
    return rollThreshold(50) ? randomInt(1, maxIndex) : 0;
}

int ComputerDecisionMaker::decideMortgageChoice(int maxIndex) {
    if (maxIndex <= 0) return 0;
    return rollThreshold(50) ? randomInt(1, maxIndex) : 0;
```

```
}

int ComputerDecisionMaker::decideJailChoice() {
    return randomInt(1, 2);
}

int ComputerDecisionMaker::decideLassoTarget(int maxIndex) {
    if (maxIndex <= 0) return 0;
    return randomInt(1, maxIndex);
}

int ComputerDecisionMaker::decideDemolitionTarget(int maxIndex) {
    if (maxIndex <= 0) return 0;
    return randomInt(1, maxIndex);
}
```

ComputerDecisionMaker adalah class yang didelegasikan dari Player dan bertanggung jawab menentukan keputusan otomatis untuk computer player. Class ini tidak menyimpan state game, tetapi menyediakan method statis untuk memilih aksi berdasarkan kondisi tertentu atau probabilitas acak. Class ini bertanggung jawab untuk menentukan command utama computer player, misalnya LEMPAR_DADU, apakah membeli properti, menentukan pilihan pajak, lelang, likuidasi, kartu, bangun properti, gadai, tebus, jail, dan target kartu, serta menyediakan helper random seperti randomInt() dan rollThreshold() untuk pengambilan keputusan.

Pendekatan delegation dipilih karena computer player memiliki data dan aturan permainan yang sama dengan pemain manusia, tetapi hanya berbeda dalam pengambilan keputusan. Dengan pendekatan ini, kode menjadi lebih sederhana dan logic bot lebih mudah dikembangkan karena cukup mengubah atau menambah method di ComputerDecisionMaker tanpa mengubah struktur utama Player.

5.1.3. Papan Dinamis

```
Rectangle GUITileController::boardTileRect(Rectangle square, int index) const {
    const int gridCells = gridCellCount();
    const int edge = gridCells - 1;
    const int perimeter = edge * 4;
    const int normalized = perimeter > 0 ? ((index % perimeter) + perimeter) % perimeter :
index;
    const float cell = square.width / static_cast<float>(gridCells);

    int row = 0;
    int column = 0;
    if (normalized <= edge) {
        row = edge;
        column = edge - normalized;
    } else if (normalized <= edge * 2) {
        row = edge * 2 - normalized;
        column = 0;
    } else if (normalized <= edge * 3) {
        row = 0;
        column = normalized - edge * 2;
    } else {
        row = normalized - edge * 3;
        column = edge;
    }
```

```
    return {square.x + column * cell, square.y + row * cell, cell, cell};
}
```

Pendekatan papan dinamis menggunakan konsep data-driven board dan polymorphic tile. Susunan papan tidak ditulis manual, tetapi dibentuk dari file konfigurasi. Data tersebut diproses oleh ConfigManager lalu digunakan Board untuk membuat berbagai jenis tile dalam satu koleksi yang seragam.

Jumlah dan isi papan bersifat fleksibel karena ditentukan dari konfigurasi, bukan angka tetap. Selain itu, tampilan (CLI/GUI) menyesuaikan ukuran dan posisi petak secara otomatis dengan perhitungan matematis berdasarkan jumlah tile sehingga tidak perlu perubahan kode saat papan berubah.

5.2. Bonus Kreasi Mandiri

5.2.1. Karakter Kebal Penjara

Bagian 13524112 Richard Samuel Simanullang

6. Pembagian Tugas

Modul (dalam poin spek)	Implementer	Tester
Application	13524010	13524052
UIManager	13524022	13524052
CommandController	13524010	13524052
GameController	13524010	13524052
TileController	13524010	13524052

Board	13524010	13524052
Dice	13524052	13524052
Game	13524052	13524052
GameContext	13524052	13524052
Player	13524062	13524052
RentContext	13524010	13524052
NimonException	13524112	13524052
InsufficientFundsException	13524112	13524052
ForcedInsufficientFundsException	13524112	13524052
CardSlotFullException	13524112	13524052
InvalidActionException	13524112	13524052
InvalidTileException	13524112	13524052
FileException	13524112	13524052
BirthdayCard	13524112	13524052
CampaignFeeCard	13524112	13524052
Card	13524112	13524052
ChanceCard	13524112	13524052
CommunityChestCard	13524112	13524052
DemolitionCard	13524112	13524052
DiscountCard	13524112	13524052
DoctorFeeCard	13524112	13524052

GoToJailCard	13524112	13524052
GoToNearestRailroadCard	13524112	13524052
HandCard	13524112	13524052
InstantCard	13524112	13524052
LassoCard	13524112	13524052
MoveBackThreeCard	13524112	13524052
MoveCard	13524112	13524052
ShieldCard	13524112	13524052
TeleportCard	13524112	13524052
Tile	13524010	13524052
ActionTile	13524010	13524052
CardTile	13524010	13524052
ChanceTile	13524010	13524052
CommunityChestTile	13524010	13524052
FestivalTile	13524010	13524052
FreeParkingTile	13524010	13524052
GoTile	13524010	13524052
GoToJailTile	13524010	13524052
IncomeTaxTile	13524010	13524052
JailTile	13524010	13524052
LuxuryTaxTile	13524010	13524052
OwnableTile	13524010	13524052

RailroadTile	13524010	13524052
SpecialTile	13524010	13524052
StreetTile	13524010	13524052
TaxTile	13524010	13524052
UtilityTile	13524010	13524052
AuctionManager	13524062	13524052
BankruptcyManager	13524062	13524052
CardDeck	13524112	13524052
CardManager	13524112	13524052
ConfigManager	13524022	13524052
FestivalManager	13524062	13524052
JailManager	13524062	13524052
LogManager	13524010	13524052
MovementManager	13524062	13524052
PropertyManager	13524062	13524052
SaveLoadManager	13524052	13524052
TaxManager	13524062	13524052
TurnManager	13524062	13524052
WinConditionManager	13524062	13524052
ActionTileConfig	13524010	13524052
PropertyConfig	13524010	13524052
LogEntry	13524010	13524052

Lampiran

Lampiran A. Form Asistensi

https://docs.google.com/document/d/1aIhVeHl7piUu8RiJe_3gnvVrHiI97kF4/edit

Lampiran B. Hasil Analisis Class dari Doxygen

https://drive.google.com/file/d/1BHEZPF_Zi9I51827z8j6tjx8AMIAZC8y/view?usp=sharing